ECPFL: Efficient Ciphertext Prefetching for Secure Cross-Device Federated Learning

Zhongyang Shi, Hong Zhong, Qingyang Zhang, Ke Cheng, Fengqun Wang, and Jie Cui

***Abstract*—Cross-device federated learning (FL) suffers from substantial communication overhead between a central server and heterogeneous, bandwidth-constrained edge devices. Although upstream sparsification and downstream prefetching can alleviate this burden, integrating them into homomorphically encrypted FL introduces two challenges: dynamically sparsified updates disrupt the spatial alignment required for CKKS SIMD aggregation, while client dropouts compromise the correctness of aggregate decryption. To address these challenges, we propose ECPFL, a dropout-resilient secure FL framework that jointly optimizes upstream and downstream communication without requiring peer-to-peer communication or decryption-share uploads during training. On the upstream side, Efficient Sparse Packing (ESP) employs a Shamir zero-sharing-based secure voting mechanism to align selected parameter packs across clients and a dropout compensation mechanism to preserve correct encrypted aggregation and one-step decryption. On the downstream side, Efficient Ciphertext Prefetching (ECP) enables intermittently available clients to asynchronously prefetch and homomorphically accumulate sparse encrypted model updates across rounds. Upon resuming training, clients recover the accumulated updates through one local decryption per pack without exposing plaintexts to the server. Experiments on MNIST, Fashion-MNIST, CIFAR-10, and SVHN demonstrate that ECPFL maintains competitive accuracy while reducing online communication volume by up to $10.7\times$ and accelerating end-to-end training by up to $5.7\times$ over the evaluated baselines.**

***Index Terms*—Federated learning, homomorphic encryption, ciphertext prefetching, sparse packing, voting mechanism, dropout resilience**

I. Introduction

FEDERATED learning (FL) has become a popular way to train shared models without moving raw data off personal devices [1]. Extending FL to the edge, cross-device FL connects millions of smartphones, wearables, and Internet of Things (IoT) devices, letting each device help train the model locally. By processing data where it is generated, this method is well-suited to privacy-sensitive applications and has driven progress in areas like healthcare [2], finance [3], and smart edge computing [4]. However, as shown in Fig. 1, cross-device FL must coordinate heterogeneous edge devices over unstable, bandwidth-constrained links and non-IID local datasets [5], [6], [7].

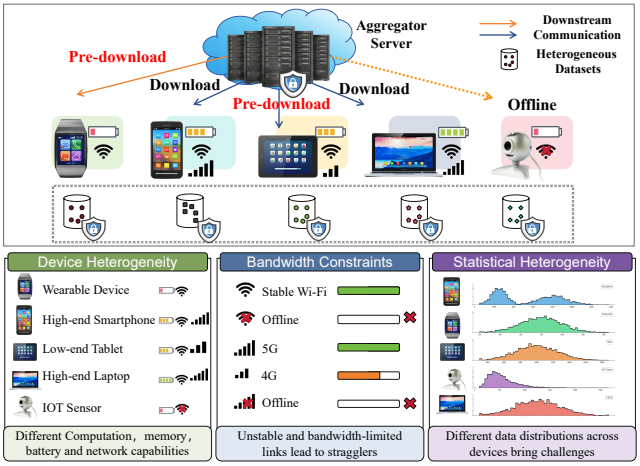


Fig. 1. **System model of heterogeneous cross-device FL.** The lower panels detail the device, bandwidth, and statistical heterogeneities across edge devices, while the upper topology illustrates parallel ciphertext prefetching.

A primary bottleneck in cross-device FL is bidirectional communication overhead between edge devices and the server [8], [9]. To ease this burden, two main strategies have been studied: sampling only a portion of devices to join in each round [10], [11], [12], [13], [14], [15], and compressing the updates using techniques like quantization [16], sparsification [17], low-rank decomposition [18], or sketching [19]. Yet, recent studies [20] show that if device sampling is combined with compressed downloads, the benefits can disappear: devices that skip some rounds end up with outdated models and must download large updates to catch up, making the total communication almost as large as full-model transmission after a few rounds.

Privacy leakage is another significant challenge in cross-device FL. Studies show that exchanging gradients or model updates does not ensure privacy, as adversaries can reconstruct sensitive information from these updates [21], [22], [23], [24]. To counteract this, three main privacy-preserving strategies have emerged: differential privacy (DP) [25], secure multi-party computation (SMPC) [26], and homomorphic encryption (HE) [27]. HE, in particular, provides strong cryptographic protection for secure aggregation by enabling computation on encrypted data. The Cheon-Kim-Kim-Song (CKKS) scheme [28], built on the Ring Learning With Errors (RLWE) assumption [29], is well-suited for machine learning tasks, supporting approximate arithmetic and Single Instruction Multiple Data (SIMD) packing. Utilizing SIMD

Zhongyang Shi, Hong Zhong, Qingyang Zhang, Fengqun Wang, and Jie Cui are with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei 230039, China, and the Anhui Engineering Laboratory of IoT Security Technologies, Anhui University, Hefei 230039, China (e-mail: zhongh@ahu.edu.cn; cuijie@mail.ustc.edu.cn). (Corresponding author: Hong Zhong.)

Ke Cheng is with the School of Computer Science and Technology, Xidian University, Xi'an, Shaanxi 710126, China, and also with the Shaanxi Key Laboratory of Network and System Security, Xi'an, Shaanxi 710071, China (e-mail: chengke@xidian.edu.cn).

packing, CKKS minimizes per-ciphertext costs and increases throughput compared to slot-wise encryption [30]. Recent advances [11], [31] have adopted CKKS packing for efficient secure aggregation, reducing upstream ciphertext size through sparsification and polynomial slot encoding. Multi-key HE (MKHE) extensions further allow aggregation across ciphertexts encrypted with different client keys, without requiring server-side decryption [32].

However, existing HE-based FL systems focus predominantly on upstream compression, treating the downstream transmission of encrypted global models as an inherent and unavoidable cost. Plaintext FL frameworks mitigate this latency by prefetching updates during idle periods, but extending this mechanism to the ciphertext domain introduces fundamental algebraic constraints: packed homomorphic ciphertexts are indivisible, so encrypted payloads cannot be partitioned or transmitted in fragments without severe dimensional expansion and decryption overhead. This rigidity creates a downstream transmission bottleneck, which is further compounded by the structural misalignment and dropout sensitivity of upstream sparsification. Specifically, pack-level sparsification [33], [34] disrupts the SIMD slot alignment required for joint decryption in MKHE, while upload-phase client dropouts—even after active clients are confirmed in the voting phase—leave missing key shares that break Lagrange interpolation and compromise the correctness of aggregate decryption. Existing remedies rely on Trusted Execution Environments (TEEs) [32], online auxiliary servers [35], or interactive decryption-share uploads, all of which introduce extra trust assumptions or communication bottlenecks.

To address these limitations, we propose ECPFL, a dropout-resilient and communication-efficient secure FL framework that extends one-step aggregate decryption to client subsampling and upload-phase dropout scenarios through two integrated mechanisms: Efficient Sparse Packing (ESP) and Efficient Ciphertext Prefetching (ECP). By incorporating Shamir secret sharing over cyclotomic rings into the additive homomorphic encryption and aggregation process [31], ECPFL eliminates peer-to-peer client communication and decryption-share uploads during training. Upstream, ESP integrates a Shamir zero-sharing-based secure voting mechanism to align selected parameter packs across clients with a dropout-resilient compensation mechanism in which surviving clients locally supply compensation ciphertexts to restore the exact aggregate encryption key contribution. Downstream, as depicted in Fig. 1, ECP exploits the additive homomorphism of encrypted model updates to allow intermittently available clients to asynchronously prefetch and accumulate sparse ciphertexts across rounds during idle periods, recovering the accumulated updates through one local decryption per pack.

We summarize our core contributions as follows:

- We propose ECPFL, a secure federated learning framework that jointly optimizes upstream and downstream communication under client subsampling and upload-phase dropouts. By incorporating Shamir secret sharing into the encryption and aggregation process, ECPFL extends one-step aggregate decryption to intermittent participation without uploading decryption shares or requiring peer-to-peer communication during training.
- We develop ESP, which enables sparse ciphertext aggregation under heterogeneous participation. A Shamir zero-sharing-based voting mechanism aligns the parameter packs selected across clients, while a compensation mechanism restores the exact aggregate encryption key contribution after dropouts. By transmitting only selected encrypted packs, ESP converts parameter sparsity into actual upstream communication savings.
- We introduce ECP, which accelerates synchronization for intermittently available clients. By exploiting additive homomorphism, clients asynchronously prefetch and accumulate sparse encrypted model updates across rounds during idle periods. Upon resuming training, they recover the accumulated updates through one local decryption per pack, reducing synchronization latency without revealing plaintexts to the server.
- We evaluate ECPFL on four datasets under heterogeneous and non-IID settings. ECPFL maintains accuracy within a small margin of the dense baseline while reducing the online communication volume by up to $10.7\times$ and the end-to-end training time by up to $5.7\times$ compared with state-of-the-art baselines.

II. Related Work

We review the related work from the following five aspects:

Multi-Key Homomorphic Encryption in FL: HE provides a mathematically rigorous foundation for privacy-preserving FL. Single-key CKKS schemes, such as FedSHE [36] and FedPHE [11], support approximate arithmetic and ciphertext packing, but rely on a globally shared secret key, leaving the system vulnerable to honest-but-curious clients. MKHE mitigates this risk by enabling independent key pairs per client [37], [38], [39]. For example, ESAFL [31] encodes packed message polynomials into a single ring element and achieves one-step decryption without uploading decryption shares, but as a cross-silo framework it fundamentally requires full client participation. In contrast, MASER [40] aligns sparse masks via majority voting yet still requires decryption-share uploads, while FedOut [35] achieves dropout resilience through two non-colluding online servers. Overall, existing MKHE schemes either mandate full participation or incur interactive decryption-share overhead, failing to simultaneously support client subsampling, upload-phase client dropouts, and non-interactive one-step decryption in cross-device FL.

Threshold Homomorphic Encryption in FL: Threshold HE employs Shamir secret sharing to distribute secret keys, allowing any qualified client subset to restore missing key contributions. Mouchet *et al.* [41] instantiated RLWE threshold access structures over $\mathcal{R}_q$ using constant integer evaluation points, while ATASSES [42] and CDKS-LSS [43] studied approximate secret sharing and exceptional sequences over cyclotomic rings to guarantee invertible Lagrange interpolation. More recently, Colin de Verdière *et al.* [44] formalized synchronous threshold FHE and proved that unmasked Lagrange weighting leaks secret keys across different participant sets; Kim *et al.* [45] then mitigated this leakage using ring-based

Shamir zero-sharing in the asynchronous setting. However, these schemes all perform key restoration during interactive online decryption, which exposes the plaintext aggregate to the server and incurs additional communication rounds. Embedding masked threshold key-contribution restoration directly into upstream encryption for non-interactive one-step decryption remains unexplored.

Gradient Compression: To alleviate the communication bottleneck in FL, various gradient compression techniques have been proposed, including quantization [46], [47], low-rank decomposition [48], [49], and sparsification [50], [51], [52]. Among these, Top-k sparsification with residual compensation [53] has proven highly effective. For encrypted FL, DSVFL [54] introduced FTop-k, utilizing an accumulable residual buffer to recover accuracy. However, DSVFL applies sparsification before Paillier encryption, strictly limiting downstream homomorphic operations to simple addition, whereas heterogeneous top-index selections across clients destroy SIMD slot alignment in packed CKKS ciphertexts and leak local gradient structures [17], [55].

Downstream Prefetching: In cross-device environments with high heterogeneity, synchronizing stale local models often offsets the bandwidth benefits achieved through client sampling and compression. To address this synchronization delay, FedFetch [20] introduces an adaptive downstream prefetching strategy in which the server presamples future participants with over-commitment and schedules background downloads of compressed updates during client idle windows. However, FedFetch operates exclusively on plaintext updates, only handles pre-round unavailability rather than upload-phase client dropouts after active-set confirmation, and cannot directly accumulate or decrypt packed multi-round ciphertexts without severe noise accumulation and repeated decryption latency.

Client Selection: Client selection is the strategy by which the server chooses the active participating subset at each round, critically determining both convergence speed and training time in cross-device FL. Uniform Random selection [56] ignores system heterogeneity and allows slow clients to stall training; recent frameworks therefore introduce targeted selection criteria, including Power-of-Choice (PoC) [10], tier-based selection (TiFL) [12], heterogeneity-aware clustered selection (HACCS) [13], and utility-guided selection (Oort) [14]. Furthermore, GlueFL [15] employs a Sticky sampling strategy that prioritizes recently active clients with fresh local models, while FedPHE [11] integrates CKKS with locality-sensitive hashing (LSH)-based Sketch-Clustering. Yet, biased selection can skew global representation under non-independent and identically distributed (non-IID) data, and existing encrypted selection methods still force newly sampled clients to synchronously download full encrypted models upon joining.

III. Preliminaries and Motivation

A. Notations

Table I summarizes the primary notations used to formalize the ECPFL architecture. In the context of our SIMD homomorphic operations, we use the terms *pack* and *batch* interchangeably to represent a vector of scalars embedded into a single polynomial ring element.

B. Cross-Device Federated Learning

In cross-device FL, each client i in the registered client set $\mathcal{C} = [N] = \{1, \ldots, N\}$ holds a private local dataset $\mathcal{D}_i$ of size $n_i = |\mathcal{D}_i|$ and trains a model with parameters $\mathbf{w} \in \mathbb{R}^d$. At round t, the server confirms $K = |\mathcal{S}^{(t)}|$ active clients in the confirmed active set $\mathcal{S}^{(t)} \subseteq \mathcal{C}$ from the presampled candidate pool $\mathcal{P}^{(t)}$, with unselected inactive set $\bar{\mathcal{S}}^{(t)} = \mathcal{C} \backslash \mathcal{S}^{(t)}$ and surviving uploader set $\widehat{\mathcal{S}}^{(t)} = \mathcal{S}^{(t)} \setminus \mathcal{D}^{(t)}$ after upload-phase dropouts $\mathcal{D}^{(t)}$. Starting from the global model $\mathbf{W}^{(t)}$ (synchronized locally as $\mathbf{w}_i$), each active client $i \in \mathcal{S}^{(t)}$ runs $\mathcal{E}$ local optimization epochs on $\mathcal{D}_i$ to obtain its local model $\mathbf{w}_i^{(t)}$ and computes its local model increment $\Delta \mathbf{w}_i^{(t)} = \mathbf{w}_i^{(t)} - \mathbf{w}_i$. This state-subtraction formulation abstracts away optimizer-specific internal states and enables pure additive synchronization. Upon confirming $\mathcal{S}^{(t)}$ in Phase 1, each active client $i \in \mathcal{S}^{(t)}$ is assigned a fixed aggregation weight $\omega_i^{(t)} = n_i / \sum_{u \in \mathcal{S}^{(t)}} n_u$ satisfying $\sum_{i \in \mathcal{S}^{(t)}} \omega_i^{(t)} = 1$. The server aggregates the weighted increments from the surviving uploader set $\widehat{\mathcal{S}}^{(t)}$ to update the global model:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} + \sum_{i \in \widehat{\mathcal{S}}^{(t)}} \omega_i^{(t)} \Delta \mathbf{w}_i^{(t)}, \tag{1}$$

where $\omega_i^{(t)}$ remains fixed after participation confirmation; if upload-phase client dropouts occur ($\widehat{\mathcal{S}}^{(t)} \subset \mathcal{S}^{(t)}$), the surviving weights are not renormalized.

C. CKKS Encoding and Additive Encryption

Our scheme builds upon CKKS slot encoding [28] and the RLWE multi-key additively homomorphic encryption construction of ESAFL [31], integrated with Shamir secret sharing over cyclotomic rings [57], [41], [44] for dropout-resilient secure aggregation.

1) Ring Learning With Errors (RLWE): The security of CKKS relies on the decisional RLWE (dRLWE) assumption [29]. Let $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$ denote the cyclotomic ring where the ring degree n is a power of two, and let $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$ denote the ciphertext polynomial ring modulo q. We define χ_s and χ_e as error distributions over $\mathcal{R}$ that output small polynomials with infinity norms strictly bounded by B_s and B_e, respectively, and let $\mathcal{U}(X)$ denote the uniform distribution over a finite set X. The dRLWE assumption states that paired samples $(\mathbf{b}, \mathbf{a}) \in \mathcal{R}_q^2$, formulated as $\mathbf{b} = [-\mathbf{a} \cdot s + e]_q$ where the polynomial $\mathbf{a} \leftarrow \mathcal{U}(\mathcal{R}_q)$ is uniform alongside a secret $s \leftarrow \chi_s$ and an error $e \leftarrow \chi_e$, are computationally indistinguishable from truly random pairs sampled uniformly from $\mathcal{U}(\mathcal{R}_q^2)$.

2) CKKS Encoding and Slot Packing: CKKS utilizes an isometric ring isomorphism $\varphi : \mathbb{R}[X]/(X^n + 1) \to \mathbb{C}^{n/2}$ to map real or complex valued vectors into integral coefficient polynomials. This mapping enables slot packing, where the encoding function $\mathrm{Ecd}(\cdot, \Delta)$ transforms a vector $\mathbf{z} \in \mathbb{C}^{n/2}$ into an encoded plaintext polynomial $m(X) \in \mathcal{R}_q$ using the CKKS scaling factor $\Delta > 1$:

$$m(X) = \mathrm{Ecd}(\mathbf{z}, \Delta) = \lfloor \Delta \cdot \varphi^{-1}(\mathbf{z}) \rceil \in \mathcal{R}_q. \tag{2}$$

This approach facilitates SIMD batching by packing $n/2$ independent scalar values into a single ciphertext (partitioning

TABLE I SUMMARY OF CORE NOTATIONS

Notation	Description	Notation	Description
Client Sets, Keys, & Secret Sharing		*Model Updates, Weights, & Ciphertexts*	
$\mathcal{C}, N$	Registered client set $[N]$ and total client count	$\mathbf{W}^{(t)}, \mathbf{w}_i^{(t)}$	Global model and local model of client i at round t
$\mathcal{P}^{(t)}, \mathcal{S}^{(t)}$	Presampled candidate pool and confirmed active set	$\Delta\mathbf{w}_i^{(t)}, \mathbf{r}_i^{(t)}$	Local model increment and unweighted residual buffer
$\bar{\mathcal{S}}^{(t)}, \mathcal{D}^{(t)}$	Unselected inactive set and upload-phase dropout set	$\mathbf{x}_i^{(t)}, \tilde{\mathbf{g}}_i^{(t)}$	Unweighted compensated update and weighted update
$\widehat{\mathcal{S}}^{(t)}, \tau$	Surviving uploader set and Shamir reconstruction threshold	$\omega_i^{(t)}, J$	Fixed aggregation weight and total pack count
$\mathcal{R}_q, \Delta$	Ciphertext polynomial ring and CKKS scaling factor	$\mathbf{v}_i^{(t)}, \hat{v}_i^{(t)}$	Binary vote bitmap and masked vote polynomial
S, s_i	Aggregate decryption key and secret key of client i	$V^{(t)}(X), \mathcal{K}^{(t)}$	Aggregated vote polynomial and consensus pack mask
σ_a, σ_b	Pseudorandom seeds for public and masking polynomials	$\mathrm{pt}_{i,j}^{(t)}, \mathrm{ct}_{i,j}^{(t)}$	Encoded plaintext polynomial and pack-j ciphertext
$a_j^{(t)}, b_j^{(t)}$	Pack-j public polynomial and zero-share masking polynomial	$\tilde{s}_{i,j}^{(t)}, \delta_{i,j}^{(t)}$	Effective encryption key and dropout compensation ciphertext
$s_{k,i}, z_i$	Shamir share of s_k at client i and zero-share of client i	$C_j^{(t)}, \mathrm{pt}_j^{(t)}$	Aggregated ciphertext and plaintext polynomial for pack j
$\lambda_i^{(t)}, \widehat{\lambda}_i^{(t)}$	Lagrange interpolation weights over $\mathcal{S}^{(t)}$ and $\widehat{\mathcal{S}}^{(t)}$	$C_{\mathrm{pre},j}, \tilde{\mathbf{g}}_{\mathrm{pre}}$	Cross-round accumulated ciphertext and prefetched update

a d-dimensional model into a total pack count of $J = \lceil 2d/n \rceil$ packs) while supporting coefficient-wise embedding for binary vote bitmaps. Finally, the decoding function $\mathrm{Dcd}(\cdot, \Delta)$ approximately recovers the encoded vector $\mathbf{z}$ via:

$$\mathbf{z} = \mathrm{Dcd}(m(X), \Delta) = \Delta^{-1} \cdot \varphi(m(X)). \tag{3}$$

3) Additive Encryption and Secret Sharing: ECPFL builds upon an RLWE-based multi-key additively homomorphic encryption (AHE) scheme [31] consisting of four core algorithms:

- $\mathbf{HE.KeyGen}(\lambda) \to (\{s_i\}_{i\in[N]}, S, \sigma_a, \sigma_b)$: Given the security parameter λ, the key generation algorithm samples N secret keys $s_i \leftarrow \mathcal{U}(\mathcal{R}_q)$, computes the aggregate decryption key $S = \left[\sum_{i=1}^{N} s_i\right]_q$, and generates pseudorandom seeds σ_a, σ_b for public and masking polynomials.
- $\mathbf{HE.Enc}(a_j^{(t)}, s_i, \mathrm{pt}_{i,j}^{(t)}) \to \mathrm{ct}_{i,j}^{(t)}$: At round t, client i encrypts the j-th encoded plaintext polynomial $\mathrm{pt}_{i,j}^{(t)}$ using its secret key s_i and the pack-j public polynomial $a_j^{(t)} = \mathrm{HE.PRPG}(\sigma_a, t \parallel j)$ derived from the pseudorandom polynomial generator (HE.PRPG):

$$\mathrm{ct}_{i,j}^{(t)} \leftarrow \mathrm{HE.Enc}\big(a_j^{(t)}, s_i, \mathrm{pt}_{i,j}^{(t)}\big) = \left[a_j^{(t)} \cdot s_i + \tilde{p} \cdot e_{i,j}^{(t)} + \mathrm{pt}_{i,j}^{(t)}\right]_q, \tag{4}$$

  where $e_{i,j}^{(t)} \leftarrow \chi_e$, $\tilde{p}$ is an integer multiple of the plaintext modulus p ($\gcd(p, q) = 1$), and $[\cdot]_q$ denotes centered reduction modulo q into $(-q/2, q/2]$.
- $\mathbf{HE.Eval}(\{\mathrm{ct}_{i,j}^{(t)}\}_{i\in[N]}) \to C_j^{(t)}$: The evaluation algorithm sums N pack-j ciphertexts to output the aggregated ciphertext $C_j^{(t)}$:

$$C_j^{(t)} \leftarrow \mathrm{HE.Eval}\Big(\big\{\mathrm{ct}_{i,j}^{(t)}\big\}_{i\in[N]}\Big) = \left[\sum_{i=1}^{N} \mathrm{ct}_{i,j}^{(t)}\right]_q. \tag{5}$$

  Substituting Eq. (4) into Eq. (5) yields:

$$\begin{aligned} C_j^{(t)} &= \left[\sum_{i=1}^{N}\Big(a_j^{(t)} \cdot s_i + \tilde{p} \cdot e_{i,j}^{(t)} + \mathrm{pt}_{i,j}^{(t)}\Big)\right]_q \\ &= \left[a_j^{(t)} \sum_{i=1}^{N} s_i + \tilde{p} \sum_{i=1}^{N} e_{i,j}^{(t)} + \sum_{i=1}^{N} \mathrm{pt}_{i,j}^{(t)}\right]_q \\ &= \left[a_j^{(t)} \cdot S + \tilde{p} \cdot E_j^{(t)} + \mathrm{pt}_j^{(t)}\right]_q, \end{aligned} \tag{6}$$

  where $E_j^{(t)} = \sum_{i=1}^{N} e_{i,j}^{(t)}$ and $\mathrm{pt}_j^{(t)} = \sum_{i=1}^{N} \mathrm{pt}_{i,j}^{(t)}$ denote the aggregated error and plaintext polynomial for pack j.
- $\mathbf{HE.Dec}(a_j^{(t)}, S, C_j^{(t)}) \to \mathrm{pt}_j^{(t)}$: Any authorized client holding (S, σ_a) recovers $\mathrm{pt}_j^{(t)}$ in a single step without uploading decryption shares:

$$\begin{aligned} \mathrm{pt}_j^{(t)} \leftarrow \mathrm{HE.Dec}\big(a_j^{(t)}, S, C_j^{(t)}\big) &= \Big[\big[C_j^{(t)} - a_j^{(t)} \cdot S\big]_q\Big]_p \\ &= \Big[\big[\tilde{p} \cdot E_j^{(t)} + \mathrm{pt}_j^{(t)}\big]_q\Big]_p = \mathrm{pt}_j^{(t)}, \end{aligned} \tag{7}$$

  where $[\cdot]_{q,p} = [[\cdot]_q]_p$ denotes centered reduction modulo q followed by reduction modulo p. Whenever $\|\tilde{p} \cdot E_j^{(t)} + \mathrm{pt}_j^{(t)}\|_\infty < q/2$, the inner reduction modulo q causes no wrap-around and the outer reduction modulo p removes $\tilde{p} \cdot E_j^{(t)}$, after which $\tilde{\mathbf{g}}_j^{(t)} = \mathrm{Dcd}(\mathrm{pt}_j^{(t)}, \Delta)$ recovers the aggregated update vector.

Cross-Pack Vulnerability. The original construction in [31] cannot be directly applied to multi-pack FL because reusing a single round-level polynomial $a^{(t)}$ across packs $j_1 \neq j_2 \in [J]$ allows any curious client holding σ_a to cancel $a^{(t)} \cdot s_i$ via $[\mathrm{ct}_{i,j_1}^{(t)} - \mathrm{ct}_{i,j_2}^{(t)}]_q = [\tilde{p}(e_{i,j_1}^{(t)} - e_{i,j_2}^{(t)}) + (\mathrm{pt}_{i,j_1}^{(t)} - \mathrm{pt}_{i,j_2}^{(t)})]_q$, exposing $(\mathrm{pt}_{i,j_1}^{(t)} - \mathrm{pt}_{i,j_2}^{(t)}) \bmod p$. We therefore derive an independent pack-j polynomial $a_j^{(t)} = \mathrm{HE.PRPG}(\sigma_a, t \parallel j)$ per pack ($a_{j_1}^{(t)} \neq a_{j_2}^{(t)}$) to eliminate cross-pack key cancellation.

Shamir Secret Sharing over $\mathcal{R}_q$ [57], [41], [44]. Client subsampling ($\mathcal{S}^{(t)} \subset \mathcal{C}$) and upload-phase client dropouts ($\mathcal{D}^{(t)} \neq \emptyset$) leave missing key contributions $\{s_k\}_{k\notin\widehat{\mathcal{S}}^{(t)}}$, compromising aggregate decryption under S. Following [41], we set constant evaluation points $\alpha_i = i \in \mathcal{R}_q$ ($i \in [N]$) under an RNS modulus $q = \prod_{\ell=1}^{L} q_\ell$ with prime factors $q_\ell > N$, ensuring $\gcd(\alpha_i - \alpha_u, q) = 1$ and $(\alpha_i - \alpha_u)^{-1} \in \mathcal{R}_q^{\times}$ for all $i \neq u$. For threshold τ, each secret key s_k is shared via a degree-$(\tau-1)$ polynomial $F_k(x)$ with $F_k(0) = s_k$ to yield shares $s_{k,i} = F_k(\alpha_i)$, while a degree-$(\tau-1)$ polynomial $\Psi(x)$ with $\Psi(0) = 0$ yields zero-shares $z_i = \Psi(\alpha_i)$. For any subset $\mathcal{S}^{(t)} \subseteq \mathcal{C}$ with $|\mathcal{S}^{(t)}| \geq \tau$, the Lagrange weights $\lambda_i^{(t)}$ over $\mathcal{S}^{(t)}$ are:

$$\lambda_i^{(t)} = \prod_{\substack{u\in\mathcal{S}^{(t)} \\ u\neq i}} (-\alpha_u)(\alpha_i - \alpha_u)^{-1} \pmod q, \tag{8}$$

satisfying the exact ring interpolation identities at $x = 0$:

$$\Big[\sum_{i\in\mathcal{S}^{(t)}} \lambda_i^{(t)} s_{k,i}\Big]_q = F_k(0) = s_k, \quad \Big[\sum_{i\in\mathcal{S}^{(t)}} \lambda_i^{(t)} z_i\Big]_q = \Psi(0) = 0. \tag{9}$$

D. Gradient Prefetching in FL

Prefetching mitigates communication latency in FL by strategically shifting downstream payload transfers to idle bandwidth periods. FedFetch [20] introduces a plaintext prefetching framework where an inactive client scheduled to re-participate after R rounds downloads the plaintext global model updates $\Delta\mathbf{W}^{(t+r)} = \mathbf{W}^{(t+r+1)} - \mathbf{W}^{(t+r)}$ during idle periods. Extending this paradigm to the ciphertext domain requires inactive clients to prefetch and homomorphically accumulate the aggregated ciphertexts $C^{(t+r)}$ over the ciphertext polynomial ring $\mathcal{R}_q$:

$$C_{\text{pre}} = \left[\sum_{r=0}^{R-1} C^{(t+r)}\right]_q. \tag{10}$$

The additive closure of $\mathcal{R}_q$ enables bounded-dimension homomorphic summation, avoiding ciphertext expansion.

E. Motivation and Challenges

While plaintext frameworks like FedFetch [20] mitigate downstream latency by allowing clients to asynchronously prefetch compressed server updates, transmitting these unencrypted payloads inherently compromises privacy guarantees. Furthermore, attempting to realize end-to-end bidirectional efficiency in the MKHE domain through the pairing of upstream update sparsification and downstream ciphertext prefetching poses three structural challenges:

- **Q1 (Alignment Dilemma):** *Heterogeneous clients tend to select different important parameters, resulting in misaligned sparse indices. How can we align their selections to enable correct pack-wise ciphertext aggregation without leaking individual selection profiles?*
- **Q2 (Dropout Dilemma):** *Partial client sampling and upload-phase client dropouts leave missing key shares in multi-key aggregation, while unmasked Lagrange interpolation across varying participant subsets leaks secret keys. How can we restore the exact aggregate key contribution $a_j^{(t)}S$ across both synchronous and asynchronous participation settings without interactive decryption rounds?*
- **Q3 (Sparsity and Prefetching Paradox):** *CKKS ciphertext size is independent of plaintext sparsity, so sparse updates still incur full communication cost. How can we redesign pack-level packing and prefetching to turn algorithmic sparsity into real bandwidth savings with one-step local decryption?*

To address these challenges, we present ECPFL, combining ESP and ECP. ESP tackles Q1 and Q2 through a Shamir zero-sharing-based secure voting mechanism and a dropout-resilient and secure pack-level sparse ciphertext upload mechanism (Section IV-C), and ECP tackles Q3 via pack-level prefetching and cross-round homomorphic accumulation (Section IV-D).

IV. Scheme Design

A. Overview

Over T rounds, following FedFetch [20], the server pre-samples a candidate pool $\mathcal{P}^{(t)} \subseteq \mathcal{C}$ of size $\lceil \beta K \rceil$ (with over-commitment factor $\beta \geq 1$) R rounds in advance to absorb pre-round stragglers. At round t, $K = |\mathcal{S}^{(t)}| \geq \tau$ responsive candidates form the confirmed active set $\mathcal{S}^{(t)} \subseteq \mathcal{P}^{(t)}$ for Phase-1 voting (with unselected inactive set $\bar{\mathcal{S}}^{(t)} = \mathcal{C} \setminus \mathcal{S}^{(t)}$), while $\widehat{\mathcal{S}}^{(t)} \subseteq \mathcal{S}^{(t)}$ ($|\widehat{\mathcal{S}}^{(t)}| \geq \tau$) denotes the surviving uploader set that completes Phase-2 upload after upload-phase dropouts $\mathcal{D}^{(t)} = \mathcal{S}^{(t)} \setminus \widehat{\mathcal{S}}^{(t)}$, satisfying $\widehat{\mathcal{S}}^{(t)} \subseteq \mathcal{S}^{(t)} \subseteq \mathcal{P}^{(t)} \subseteq \mathcal{C}$.

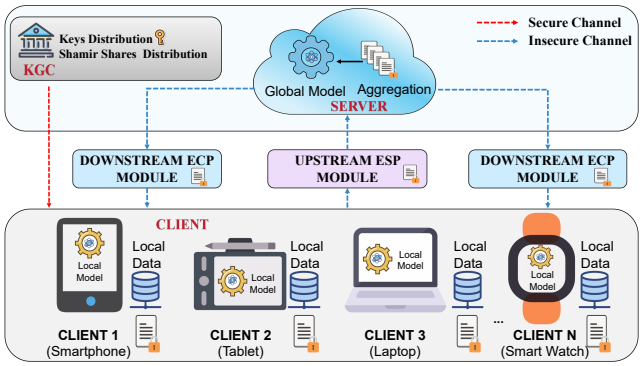


Fig. 2. **ECPFL system architecture.** The KGC distributes keys and Shamir shares once during Phase 0 setup. Clients upload sparse ciphertexts via the ESP module and prefetch aggregated ciphertexts via the ECP module.

Protocol Details. Fig. 2 illustrates the system architecture, and Alg. 1 formalizes the main procedure of ECPFL, which invokes the ESP Upstream Module (Alg. 2) and the ECP Downstream Module (Alg. 3) in chronological order:

Phase 0: System Initialization. The KGC generates and distributes $(s_i, S, \sigma_a, \sigma_b, \{s_{k,i}\}_{k=1}^N, z_i)$ to each client $i \in \mathcal{C}$ over secure channels, erases all temporary polynomials, and remains offline thereafter.

Phase 1: Pack-Level Consensus Voting. Each active client $i \in \mathcal{S}^{(t)}$ synchronizes $\mathbf{w}_i$ via ECP-Sync, trains $\mathbf{w}_i^{(t)}$ to obtain $\Delta\mathbf{w}_i^{(t)}$, and invokes ESP-Vote to form $\mathbf{x}_i^{(t)}$ and upload the masked vote $\hat{v}_i^{(t)}$, from which the server derives and broadcasts the consensus pack mask $\mathcal{K}^{(t)}$ via ESP-Tally.

Phase 2: Sparse Packs Upload. Guided by $\mathcal{K}^{(t)}$, each surviving client $i \in \widehat{\mathcal{S}}^{(t)}$ invokes ESP-Enc to weight $\tilde{\mathbf{g}}_i^{(t)} = \omega_i^{(t)}\mathbf{x}_i^{(t)}$, update its residual $\mathbf{r}_i^{(t)}$, and upload the encrypted packs $\{\text{ct}_{i,j}^{(t)}\}_{j\in\mathcal{K}^{(t)}}$ under the effective key $\tilde{s}_{i,j}^{(t)}$.

Phase 3: Homomorphic Aggregation. The server sums $\{\text{ct}_{i,j}^{(t)}\}_{i\in\mathcal{S}^{(t)}}$ directly via ESP-Agg if $\mathcal{D}^{(t)} = \emptyset$, or collects compensation ciphertexts $\{\delta_{i,j}^{(t)}\}_{j\in\mathcal{K}^{(t)}}$ from $\widehat{\mathcal{S}}^{(t)}$ via ESP-Comp before aggregating if $\mathcal{D}^{(t)} \neq \emptyset$, restoring the aggregate key contribution $a_j^{(t)}S$ in both cases.

Phase 4: Ciphertexts Prefetching. The server caches $\{C_j^{(t)}\}_{j\in\mathcal{K}^{(t)}}$ in the sliding queue $\mathcal{Q}$ via ECP-Queue, while presampled clients in $\mathcal{P}^{(t+R)}$ asynchronously fetch missed packs via ECP-Fetch during idle periods for one-step local decryption upon rejoining.

Algorithm 1: ECPFL Main Procedure

Input: $\{\mathcal{D}_i\}_{i=1}^N$, $\mathbf{W}^{(0)}$, T, R, τ
Output: Synchronized global model across clients

KGC generates $\{s_i\}_{i=1}^N, S, \sigma_a, \sigma_b, \{s_{k,i}\}_{k,i=1}^N, \{z_i\}_{i=1}^N$
KGC sends $(s_i, S, \sigma_a, \sigma_b, \{s_{k,i}\}_{k=1}^N, z_i)$ to each $i \in \mathcal{C}$
Initialize $\mathbf{w}_i \leftarrow \mathbf{W}^{(0)}, \mathbf{r}_i^{(0)} \leftarrow \mathbf{0}, t_{\text{last},i} \leftarrow 0, \mathcal{L}_{i,j}, \mathcal{Q} \leftarrow \emptyset$
for $t = 1, 2, \ldots, T$ **do**
  Select $\mathcal{S}^{(t)} \subseteq \mathcal{P}^{(t)}$, $\bar{\mathcal{S}}^{(t)} \leftarrow \mathcal{C} \setminus \mathcal{S}^{(t)}$, and $\mathcal{P}^{(t+R)}$
  Set $\omega_i^{(t)} \leftarrow n_i / \sum_{u \in \mathcal{S}^{(t)}} n_u$
  Broadcast $(\mathcal{S}^{(t)}, \bar{\mathcal{S}}^{(t)}, \omega_i^{(t)})$
  foreach *active client* $i \in \mathcal{S}^{(t)}$ *in parallel* **do**
    $\mathbf{w}_i \leftarrow$ ECP-SYNC$(i, t, \mathbf{w}_i, \{\mathcal{L}_{i,j}\})$
    Train $\mathbf{w}_i^{(t)}$ on $\mathcal{D}_i$ and set $\Delta\mathbf{w}_i^{(t)} \leftarrow \mathbf{w}_i^{(t)} - \mathbf{w}_i$
    $(\hat{v}_i^{(t)}, \mathbf{x}_i^{(t)}) \leftarrow$ ESP-VOTE$(i, t, \Delta\mathbf{w}_i^{(t)}, \mathbf{r}_i^{(t-1)})$
    Upload masked vote $\hat{v}_i^{(t)}$ to Server
  Receive $\{\hat{v}_i^{(t)}\}_{i \in \mathcal{S}^{(t)}}$ from $\mathcal{S}^{(t)}$ at Server
  Compute $\mathcal{K}^{(t)} \leftarrow$ ESP-TALLY$(\{\hat{v}_i^{(t)}\}_{i \in \mathcal{S}^{(t)}})$
  Broadcast $\mathcal{K}^{(t)}$ to active clients $\mathcal{S}^{(t)}$
  foreach *surviving client* $i \in \widehat{\mathcal{S}}^{(t)}$ *in parallel* **do**
    Receive consensus mask $\mathcal{K}^{(t)}$ from Server
    $\{\text{ct}_{i,j}^{(t)}\}_{j \in \mathcal{K}^{(t)}} \leftarrow$ ESP-ENC$(i, t, \mathbf{x}_i^{(t)}, \omega_i^{(t)}, \mathcal{K}^{(t)})$
    Upload $\{\text{ct}_{i,j}^{(t)}\}_{j \in \mathcal{K}^{(t)}}$ to Server
  if $\mathcal{D}^{(t)} = \mathcal{S}^{(t)} \setminus \widehat{\mathcal{S}}^{(t)} \neq \emptyset$ **then**
    Broadcast surviving uploader set $\widehat{\mathcal{S}}^{(t)}$
    foreach *surviving client* $i \in \widehat{\mathcal{S}}^{(t)}$ *in parallel* **do**
      $\{\delta_{i,j}^{(t)}\}_{j \in \mathcal{K}^{(t)}} \leftarrow$ ESP-COMP$(i, t, \widehat{\mathcal{S}}^{(t)}, \mathcal{K}^{(t)})$
      Upload $\{\delta_{i,j}^{(t)}\}_{j \in \mathcal{K}^{(t)}}$ to Server
  Compute $\{C_j^{(t)}\}_{j \in \mathcal{K}^{(t)}} \leftarrow$ ESP-AGG$(t, \widehat{\mathcal{S}}^{(t)}, \mathcal{K}^{(t)})$
  Update $\mathcal{Q} \leftarrow$ ECP-QUEUE$(t, \mathcal{K}^{(t)}, \{C_j^{(t)}\}, \mathcal{Q})$
  foreach *client* $i \in \mathcal{P}^{(t+R)}$ *in parallel* **do**
    $\{\mathcal{L}_{i,j}\} \leftarrow$ ECP-FETCH$(i, t, \mathcal{Q}, \{\mathcal{L}_{i,j}\})$

B. Threat Model and Security Goals

1) Threat Model: ECPFL consists of three entities with the following trust assumptions:

- **Key Generation Center (KGC).** The fully trusted KGC initializes client keys, Shamir shares, and pseudorandom seeds during Phase 0. It securely distributes $(s_i, S, \sigma_a, \sigma_b, z_i)$ and the corresponding $\mathcal{O}(N)$ shares to each client, erases temporary key-generation material, and remains offline during training.
- **Honest-but-Curious Server.** The server follows the prescribed voting, aggregation, and prefetching procedures but may attempt to infer private model updates, vote bitmaps, or global model parameters from observed messages and metadata.
- **Honest-but-Curious Clients.** Clients follow the protocol but may collude to infer honest clients' private updates, secret keys, or vote bitmaps. We consider at most $\kappa \leq \tau - 2$ colluding clients.

2) Adversarial Capabilities: We consider a semi-honest adversary $\mathcal{A}$ that may corrupt the server ($\mathcal{A}_{\text{srv}}$), up to $\kappa \leq \tau - 2$ clients ($\mathcal{A}_{\text{cli}}$), or passively eavesdrop on public communications. Server–client collusion is excluded because authorized clients hold the aggregate decryption key S. For any valid round with $|\widehat{\mathcal{S}}^{(t)}| \geq \tau$, at least two honest clients survive:

$$|\mathcal{H}^{(t)}| = |\widehat{\mathcal{S}}^{(t)} \setminus \mathcal{K}_{\text{cor}}| \geq \tau - \kappa \geq 2.$$

Secure channels are used only for KGC initialization; training communications are publicly observable. The server observes participation and consensus sets $(\mathcal{P}^{(t)}, \mathcal{S}^{(t)}, \bar{\mathcal{S}}^{(t)}, \widehat{\mathcal{S}}^{(t)}, \mathcal{K}^{(t)})$.

3) Security Goals: ECPFL is designed to achieve the following security guarantees:

- **Individual ciphertext confidentiality.** Each honest client's uploaded ciphertext $\text{ct}_{h,j}^{(t)}$ computationally conceals its local plaintext $\text{pt}_{h,j}^{(t)}$ against both $\mathcal{A}_{\text{srv}}$ and any coalition $\mathcal{A}_{\text{cli}}$ of $\kappa \leq \tau - 2$ colluding clients.
- **Server obliviousness.** Without client–server collusion, the server $\mathcal{A}_{\text{srv}}$ learns neither any individual plaintext $\text{pt}_{i,j}^{(t)}$ nor the aggregated plaintext $\text{pt}_j^{(t)}$.
- **Collusion resistance.** Any coalition $\mathcal{A}_{\text{cli}}$ of $\kappa \leq \tau - 2$ colluding clients cannot recover honest clients' keys $\{s_h\}$, zero-shares $\{z_h\}$, or individual updates $\text{pt}_{h,j}^{(t)}$, learning at most the honest sum $\sum_{h \in \mathcal{H}^{(t)}} \text{pt}_{h,j}^{(t)}$ ($|\mathcal{H}^{(t)}| \geq 2$).
- **Voting and compensation privacy.** Masked votes $\hat{v}_h^{(t)}$ conceal individual vote bitmaps $\mathbf{v}_h^{(t)}$ beyond the global tally, while compensation ciphertexts $\delta_{h,j}^{(t)}$ restore $a_j^{(t)} S$ without leaking honest secret keys or Shamir shares.
- **Cross-round prefetching privacy.** The accumulated ciphertext $C_{\text{pre},j}$ exposes no plaintext to the server and reveals no single-client update beyond the authorized round aggregates to rejoining clients.

C. ESP: Efficient Sparse Packing

To address the **Alignment Dilemma (Q1)** and the **Dropout Dilemma (Q2)** while improving aggregation efficiency under client subsampling and upload-phase dropouts, we propose the ESP mechanism. The upstream process is formalized in Alg. 2 and illustrated in Fig. 3.

1) Local Compensation and Consensus Voting: To avoid the SIMD misalignment introduced by element-wise sparsification, ESP performs sparsification at the parameter-pack level. To accommodate client unavailability before or during Phase-1 voting, ECPFL adopts the over-commitment strategy of FedFetch [20]. Specifically, the first K responsive clients among the $\lceil \beta K \rceil$ presampled candidates in $\mathcal{P}^{(t)}$ form the confirmed active set $\mathcal{S}^{(t)} \subseteq \mathcal{P}^{(t)}$, with $|\mathcal{S}^{(t)}| = K \geq \tau$. After synchronizing its local model $\mathbf{w}_i$ via ECP-SYNC and performing $\mathcal{E}$ local training epochs, each client $i \in \mathcal{S}^{(t)}$ computes its model increment $\Delta\mathbf{w}_i^{(t)} = \mathbf{w}_i^{(t)} - \mathbf{w}_i$ and incorporates the unweighted residual buffer $\mathbf{r}_i^{(t-1)}$ to obtain

$$\mathbf{x}_i^{(t)} = \Delta\mathbf{w}_i^{(t)} + \mathbf{r}_i^{(t-1)}.$$

The client partitions $\mathbf{x}_i^{(t)}$ into J parameter packs and selects the Top-ζ fraction according to their ℓ_2-norms, producing a binary vote bitmap $\mathbf{v}_i^{(t)} = (v_{i,1}^{(t)}, \ldots, v_{i,J}^{(t)}) \in \{0,1\}^J$.

Through secure vote aggregation, the server obtains the global voting score $\nu_j^{(t)} = \sum_{i \in \mathcal{S}^{(t)}} v_{i,j}^{(t)}$ for each pack j and determines the consensus pack set $\mathcal{K}^{(t)} \subseteq [J]$, represented

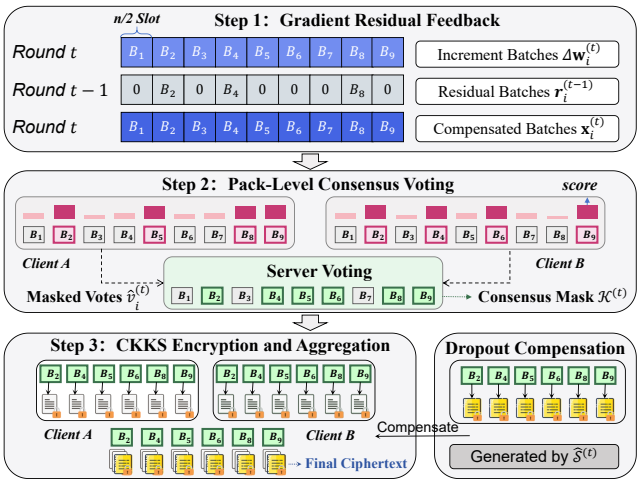


Fig. 3. **ESP upstream module.** *Step 1:* merge previous-round residuals with current increments to form compensated updates. *Step 2:* active clients vote for important packs and the server derives the consensus pack mask. *Step 3:* clients encrypt and upload only the consensus-selected packs.

equivalently by $\mathbf{k}^{(t)} \in \{0,1\}^J$, where $k_j^{(t)} = 1$ iff $j \in \mathcal{K}^{(t)}$. We consider three consensus variants:

$$\mathcal{K}^{(t)} = \begin{cases} \{j \in [J] \mid \nu_j^{(t)} \geq 1\}, & \text{(ECPFL-UN)}, \\ \mathrm{Top}_{\lceil \theta J \rceil}\big(\{\nu_j^{(t)}\}_{j=1}^J\big), & \text{(ECPFL-RK-}\theta\text{)}, \\ \{j \in [J] \mid \nu_j^{(t)} \geq \theta K\}, & \text{(ECPFL-TH-}\theta\text{)}. \end{cases} \tag{11}$$

Here, $\theta \in (0,1]$ controls the retained-pack ratio in ECPFL-RK and the voting threshold in ECPFL-TH. The operator Top_m returns the indices of the m highest voting scores, breaking ties by ascending pack index.

2) Shamir Zero-Sharing Secure Voting: Transmitting plaintext vote bitmaps $\mathbf{v}_i^{(t)}$ inherently exposes the sparse structure of local updates, enabling gradient-index and label-inference attacks [17], [55], [54], [23]. To preserve index privacy, we design a *Shamir zero-sharing-based secure voting mechanism* with single-polynomial coefficient packing. Because $J \ll n$, each active client $i \in \mathcal{S}^{(t)}$ embeds its J-bit binary vote bitmap into the lower J coefficients of $v_i^{(t)}(X) = \sum_{j=1}^J v_{i,j}^{(t)} X^{j-1} \in \mathcal{R}_p$ and generates the voting public polynomial $a_0^{(t)} = \mathrm{HE.PRPG}(\sigma_b, t \parallel 0) \in \mathcal{R}_q$ using reserved index 0 for domain separation from pack indices $j \in [J]$. Using its Lagrange interpolation weight $\lambda_i^{(t)}$ over $\mathcal{S}^{(t)}$ and a fresh error polynomial $e_{i,0}^{(t)} \leftarrow \chi_e$, client i uploads the masked vote polynomial:

$$\hat{v}_i^{(t)} = \Big[a_0^{(t)} \cdot \lambda_i^{(t)} z_i + \tilde{p} \cdot e_{i,0}^{(t)} + v_i^{(t)}(X) \Big]_q \in \mathcal{R}_q. \tag{12}$$

Upon receiving $\{\hat{v}_i^{(t)}\}_{i \in \mathcal{S}^{(t)}}$, the server sums the masked vote polynomials in $\mathcal{R}_q$ and reduces modulo p. Factoring out the common polynomial $a_0^{(t)}$ and applying the Shamir zero-sharing identity $\big[\sum_{i \in \mathcal{S}^{(t)}} \lambda_i^{(t)} z_i\big]_q = \Psi(0) = 0$ from Eq. (9)

Algorithm 2: ESP Upstream Module

Parameter: $J, q, p, \Delta, \zeta, \gamma_d, \gamma_s, (s_i, \{s_{k,i}\}_{k=1}^N, z_i, \sigma_a, \sigma_b)$

Function ESP-VOTE$(i, t, \Delta \mathbf{w}_i^{(t)}, \mathbf{r}_i^{(t-1)})$
- Compute $\mathbf{x}_i^{(t)} \leftarrow \Delta \mathbf{w}_i^{(t)} + \mathbf{r}_i^{(t-1)}$
- Form $v_i^{(t)}(X) \leftarrow \sum_{j=1}^J v_{i,j}^{(t)} X^{j-1}$
- Compute masked vote $\hat{v}_i^{(t)}$ as Eq. (12)
- **return** $(\hat{v}_i^{(t)}, \mathbf{x}_i^{(t)})$

Function ESP-TALLY$(\{\hat{v}_i^{(t)}\}_{i \in \mathcal{S}^{(t)}})$
- Compute $V^{(t)}(X)$ and scores $\{\nu_j^{(t)}\}_{j=1}^J$ as Eq. (13)
- Determine consensus pack mask $\mathcal{K}^{(t)}$ as Eq. (11)
- **return** $\mathcal{K}^{(t)}$

Function ESP-ENC$(i, t, \mathbf{x}_i^{(t)}, \omega_i^{(t)}, \mathcal{K}^{(t)})$
- Compute weighted update $\tilde{\mathbf{g}}_i^{(t)} \leftarrow \omega_i^{(t)} \mathbf{x}_i^{(t)}$
- Update residual $\mathbf{r}_i^{(t)}$ as Eq. (15) and set $t_{\mathrm{last},i} \leftarrow t$
- **foreach** $j \in \mathcal{K}^{(t)}$ **do**
  - Encode $\mathrm{pt}_{i,j}^{(t)} \leftarrow \mathrm{Ecd}(\tilde{\mathbf{g}}_{i,j}^{(t)}, \Delta)$
  - Encrypt $\mathrm{ct}_{i,j}^{(t)}$ as Eq. (14)
- **return** $\{\mathrm{ct}_{i,j}^{(t)}\}_{j \in \mathcal{K}^{(t)}}$

Function ESP-COMP$(i, t, \widehat{\mathcal{S}}^{(t)}, \mathcal{K}^{(t)})$
- Compute updated Lagrange weight $\widehat{\lambda}_i^{(t)}$ over $\widehat{\mathcal{S}}^{(t)}$
- **foreach** $j \in \mathcal{K}^{(t)}$ **do**
  - Compute $\kappa_{i,j}^{(t)}$ and $\delta_{i,j}^{(t)}$ as Eq. (17)
- **return** $\{\delta_{i,j}^{(t)}\}_{j \in \mathcal{K}^{(t)}}$

Function ESP-AGG$(t, \widehat{\mathcal{S}}^{(t)}, \mathcal{K}^{(t)})$
- **foreach** $j \in \mathcal{K}^{(t)}$ **do**
  - Compute aggregate $C_j^{(t)}$ as Eq. (16) or Eq. (19)
- **return** $\{C_j^{(t)}\}_{j \in \mathcal{K}^{(t)}}$

yields step by step:

$$\begin{aligned} V^{(t)}(X) &= \left[\left[\sum_{i \in \mathcal{S}^{(t)}} \hat{v}_i^{(t)} \right]_q \right]_p \\ &= \left[\sum_{i \in \mathcal{S}^{(t)}} \Big(a_0^{(t)} \lambda_i^{(t)} z_i + \tilde{p} e_{i,0}^{(t)} + v_i^{(t)}(X) \Big) \right]_{q,p} \\ &= \left[a_0^{(t)} \underbrace{\sum_{i \in \mathcal{S}^{(t)}} \lambda_i^{(t)} z_i}_{=\Psi(0)=0} + \tilde{p} E_0^{(t)} + \sum_{i \in \mathcal{S}^{(t)}} v_i^{(t)}(X) \right]_{q,p} \\ &= \sum_{j=1}^J \left(\sum_{i \in \mathcal{S}^{(t)}} v_{i,j}^{(t)} \right) X^{j-1} = \sum_{j=1}^J \nu_j^{(t)} X^{j-1}, \end{aligned} \tag{13}$$

where $E_0^{(t)} = \sum_{i \in \mathcal{S}^{(t)}} e_{i,0}^{(t)}$. Whenever $\|\tilde{p} E_0^{(t)} + \sum_{i \in \mathcal{S}^{(t)}} v_i^{(t)}(X)\|_\infty < q/2$ and $|\mathcal{S}^{(t)}| < p$, the server extracts the exact pack scores $\{\nu_j^{(t)}\}_{j=1}^J$ from the lower J coefficients of the aggregated vote polynomial $V^{(t)}(X)$ without leaking individual vote bitmaps.

3) Dropout-Resilient Sparse Ciphertext Upload: Upon receiving the consensus pack mask $\mathcal{K}^{(t)}$, each active client scales its unweighted compensated update by the fixed aggregation weight $\omega_i^{(t)} = n_i / \sum_{u \in \mathcal{S}^{(t)}} n_u$ to form the weighted update $\tilde{\mathbf{g}}_i^{(t)} = \omega_i^{(t)} \mathbf{x}_i^{(t)}$ and encrypts only the selected packs

$j \in \mathcal{K}^{(t)}$. For each $j \in \mathcal{K}^{(t)}$, client i encodes the j-th pack into the encoded plaintext polynomial $\mathrm{pt}_{i,j}^{(t)} = \mathrm{Ecd}(\tilde{\mathbf{g}}_{i,j}^{(t)}, \Delta)$. Client i then derives the pack-j public polynomial $a_j^{(t)} = \mathrm{HE.PRPG}(\sigma_a, t \parallel j)$ and the zero-share masking polynomial $b_j^{(t)} = \mathrm{HE.PRPG}(\sigma_b, t \parallel j)$, forms the effective encryption key $\tilde{s}_{i,j}^{(t)}$ embedding the Lagrange-weighted Shamir shares of the unselected inactive set $\bar{\mathcal{S}}^{(t)}$ masked by $b_j^{(t)} z_i$, and generates the pack-j ciphertext $\mathrm{ct}_{i,j}^{(t)}$:

$$\begin{aligned} \tilde{s}_{i,j}^{(t)} &= \Big[s_i + \lambda_i^{(t)} \sum_{k \in \bar{\mathcal{S}}^{(t)}} s_{k,i} + \lambda_i^{(t)} b_j^{(t)} z_i \Big]_q, \\ \mathrm{ct}_{i,j}^{(t)} &= \mathrm{HE.Enc}\big(a_j^{(t)}, \tilde{s}_{i,j}^{(t)}, \mathrm{pt}_{i,j}^{(t)}\big) \\ &= \Big[a_j^{(t)} \cdot \tilde{s}_{i,j}^{(t)} + \tilde{p} \cdot e_{i,j}^{(t)} + \mathrm{pt}_{i,j}^{(t)} \Big]_q. \end{aligned} \tag{14}$$

Role and Principle of the Pack-Specific Mask $b_j^{(t)} z_i$. As shown by Colin de Verdière *et al.* [44], if client i embeds the unmasked key-share sum $s_i + \lambda_i^{(t)} \sum_{k \in \bar{\mathcal{S}}^{(t)}} s_{k,i}$ across rounds with varying active sets, an adversary observing the resulting ciphertexts can assemble a system of linear equations in the public Lagrange weights $\lambda_i^{(t)}$ and recover s_i together with the shares $\{s_{k,i}\}$. Masking the combination with the zero-share z_i defeats this attack, since $\sum_{i \in \mathcal{S}^{(t)}} \lambda_i^{(t)} z_i = 0$ vanishes upon aggregation. However, a static zero-share is not sufficient on its own: z_i is generated once and reused across all packs and rounds, so an unscaled term $\lambda_i^{(t)} z_i$ enters every pack ciphertext of client i identically, and an adversary knowing the public polynomials $a_j^{(t)}$ (via seed σ_a) can form linear combinations across packs $j_1 \neq j_2$ of the same round that eliminate the common key term and expose combinations of the pack plaintexts $\mathrm{pt}_{i,j_1}^{(t)}$ and $\mathrm{pt}_{i,j_2}^{(t)}$. Scaling z_i by the fresh zero-share masking polynomial $b_j^{(t)} = \mathrm{HE.PRPG}(\sigma_b, t \parallel j)$ (domain-separated from $a_0^{(t)} = \mathrm{HE.PRPG}(\sigma_b, t \parallel 0)$) makes the effective mask $b_j^{(t)} z_i$ distinct for every pack and round, thereby preventing such cross-pack elimination, while still factoring out of the server-side sum because $b_j^{(t)}$ is common to all clients on pack j and satisfies $\sum_{i \in \mathcal{S}^{(t)}} \lambda_i^{(t)} b_j^{(t)} z_i = b_j^{(t)} \sum_{i \in \mathcal{S}^{(t)}} \lambda_i^{(t)} z_i = 0$.

Finally, the client updates its unweighted residual buffer $\mathbf{r}_i^{(t)}$ to retain every pack excluded by the consensus pack mask using the unweighted compensated update $\mathbf{x}_i^{(t)}$, ensuring that $\mathbf{r}_i^{(t)}$ is never repeatedly multiplied by $\omega_i^{(t)}$ across rounds. To avoid unbounded residual accumulation and alleviate staleness under intermittent participation, ECPFL applies dual-decay residual compensation:

$$\mathbf{r}_i^{(t)} = \gamma_d \, \gamma_s^{\ell_i^{(t)} - 1} \Big(\mathbf{x}_i^{(t)} \odot \big(\mathbf{1} - \mathbf{k}^{(t)} \big) \Big), \tag{15}$$

where $\odot$ denotes the pack-wise Hadamard product, $\mathbf{1} \in \{1\}^J$ is the all-ones vector, $\gamma_d \in (0, 1]$ is the per-round decay factor, $\gamma_s \in (0, 1]$ is the cross-round decay (CRD) factor, and $\ell_i^{(t)} = t - t_{\mathrm{last},i} \geq 1$ is the number of elapsed rounds since client i's last participation round $t_{\mathrm{last},i}$.

4) Global Homomorphic Aggregation: Following the synchronous and asynchronous threshold paradigms [44], [45], the server invokes HE.Eval in ESP-AGG to aggregate the uploaded sparse ciphertexts under two settings:

Synchronous Setting ($\mathcal{D}^{(t)} = \emptyset$, $\widehat{\mathcal{S}}^{(t)} = \mathcal{S}^{(t)}$). When all active clients confirmed in $\mathcal{S}^{(t)}$ complete their Phase-2 upload, the server directly sums $\{\mathrm{ct}_{i,j}^{(t)}\}_{i \in \mathcal{S}^{(t)}}$. Substituting Eq. (14), exchanging the double summation order over $i \in \mathcal{S}^{(t)}$ and $k \in \bar{\mathcal{S}}^{(t)}$, and applying the Shamir interpolation identities in Eq. (9) (with $\mathcal{S}^{(t)} \cup \bar{\mathcal{S}}^{(t)} = \mathcal{C}$) yields step by step:

$$\begin{aligned} C_j^{(t)} &= \mathrm{HE.Eval}\Big(\{\mathrm{ct}_{i,j}^{(t)}\}_{i \in \mathcal{S}^{(t)}} \Big) = \Big[\sum_{i \in \mathcal{S}^{(t)}} \mathrm{ct}_{i,j}^{(t)} \Big]_q \\ &= \Big[\sum_{i \in \mathcal{S}^{(t)}} \Big(a_j^{(t)} \cdot \tilde{s}_{i,j}^{(t)} + \tilde{p} \cdot e_{i,j}^{(t)} + \mathrm{pt}_{i,j}^{(t)} \Big) \Big]_q \\ &= \Big[a_j^{(t)} \Big(\sum_{i \in \mathcal{S}^{(t)}} s_i + \sum_{k \in \bar{\mathcal{S}}^{(t)}} \underbrace{\sum_{i \in \mathcal{S}^{(t)}} \lambda_i^{(t)} s_{k,i}}_{= s_k} + b_j^{(t)} \underbrace{\sum_{i \in \mathcal{S}^{(t)}} \lambda_i^{(t)} z_i}_{= 0} \Big) \\ &\qquad + \tilde{p} \sum_{i \in \mathcal{S}^{(t)}} e_{i,j}^{(t)} + \sum_{i \in \mathcal{S}^{(t)}} \mathrm{pt}_{i,j}^{(t)} \Big]_q \\ &= \Big[a_j^{(t)} S + \tilde{p} E_j^{(t)} + \mathrm{pt}_j^{(t)} \Big]_q, \end{aligned} \tag{16}$$

where $E_j^{(t)} = \sum_{i \in \mathcal{S}^{(t)}} e_{i,j}^{(t)}$ and $\mathrm{pt}_j^{(t)} = \sum_{i \in \mathcal{S}^{(t)}} \mathrm{pt}_{i,j}^{(t)}$ denote the aggregated error and plaintext polynomial for pack j.

Asynchronous Setting ($\mathcal{D}^{(t)} \neq \emptyset$, $\widehat{\mathcal{S}}^{(t)} = \mathcal{S}^{(t)} \setminus \mathcal{D}^{(t)}$). If upload-phase client dropouts occur after voting, only the surviving uploader set $\widehat{\mathcal{S}}^{(t)}$ ($|\widehat{\mathcal{S}}^{(t)}| \geq \tau$) completes the upload of $\{\mathrm{ct}_{i,j}^{(t)}\}_{i \in \widehat{\mathcal{S}}^{(t)}}$, and the stale weights $\lambda_i^{(t)}$ cannot reconstruct the missing key shares. The server broadcasts $\widehat{\mathcal{S}}^{(t)}$ to the surviving clients, and each surviving client $i \in \widehat{\mathcal{S}}^{(t)}$ computes the updated Lagrange interpolation weight $\widehat{\lambda}_i^{(t)} = \prod_{u \in \widehat{\mathcal{S}}^{(t)} \setminus \{i\}} (-\alpha_u)(\alpha_i - \alpha_u)^{-1} \bmod q$ over $\widehat{\mathcal{S}}^{(t)}$, forms the local compensation key term $\kappa_{i,j}^{(t)}$, and uploads the dropout compensation ciphertext $\delta_{i,j}^{(t)}$ with fresh error $\hat{e}_{i,j}^{(t)} \leftarrow \chi_e$:

$$\begin{aligned} \kappa_{i,j}^{(t)} &= \Big[\widehat{\lambda}_i^{(t)} \sum_{k \notin \widehat{\mathcal{S}}^{(t)}} s_{k,i} - \lambda_i^{(t)} \sum_{k \in \bar{\mathcal{S}}^{(t)}} s_{k,i} \\ &\qquad + \big(\widehat{\lambda}_i^{(t)} - \lambda_i^{(t)} \big) b_j^{(t)} z_i \Big]_q, \\ \delta_{i,j}^{(t)} &= \Big[a_j^{(t)} \cdot \kappa_{i,j}^{(t)} + \tilde{p} \cdot \hat{e}_{i,j}^{(t)} \Big]_q. \end{aligned} \tag{17}$$

Adding $\kappa_{i,j}^{(t)}$ in Eq. (17) to $\tilde{s}_{i,j}^{(t)}$ in Eq. (14) for each surviving client $i \in \widehat{\mathcal{S}}^{(t)}$ (where $k \notin \widehat{\mathcal{S}}^{(t)}$ denotes $\bar{\mathcal{S}}^{(t)} \cup \mathcal{D}^{(t)}$) cancels the stale $\lambda_i^{(t)}$ terms directly:

$$\begin{aligned} \tilde{s}_{i,j}^{(t)} + \kappa_{i,j}^{(t)} &\equiv s_i + \lambda_i^{(t)} \sum_{k \in \bar{\mathcal{S}}^{(t)}} s_{k,i} + \lambda_i^{(t)} b_j^{(t)} z_i \\ &\quad + \widehat{\lambda}_i^{(t)} \sum_{k \notin \widehat{\mathcal{S}}^{(t)}} s_{k,i} - \lambda_i^{(t)} \sum_{k \in \bar{\mathcal{S}}^{(t)}} s_{k,i} \\ &\quad + \big(\widehat{\lambda}_i^{(t)} - \lambda_i^{(t)} \big) b_j^{(t)} z_i \\ &\equiv s_i + \widehat{\lambda}_i^{(t)} \sum_{k \notin \widehat{\mathcal{S}}^{(t)}} s_{k,i} + \widehat{\lambda}_i^{(t)} b_j^{(t)} z_i \pmod q. \end{aligned} \tag{18}$$

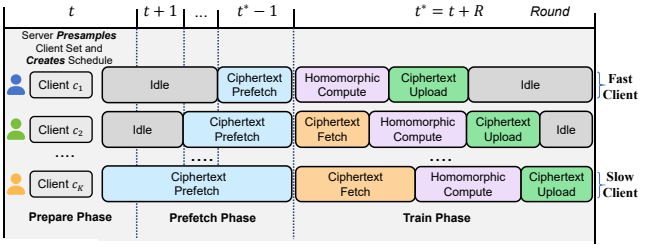


Fig. 4. **ECP downstream module.** In the Prepare Phase (round t), the server presamples candidates for round $t^* = t + R$. During the Prefetch Phase, inactive clients asynchronously accumulate sparse global ciphertexts. In the Train Phase (round $t^* = t+R$), active clients fetch any remaining packs and execute one-step local decryption.

Algorithm 3: ECP Downstream Module

Parameter: R, J, Δ, (S, σ_a)

Function ECP-QUEUE$(t, \mathcal{K}^{(t)}, \{C_j^{(t)}\}_{j\in\mathcal{K}^{(t)}}, \mathcal{Q})$
- Append $(t, \mathcal{K}^{(t)}, \{C_j^{(t)}\}_{j\in\mathcal{K}^{(t)}})$ to sliding queue $\mathcal{Q}$
- Remove $r \le t - R$ from $\mathcal{Q}$
- **return** $\mathcal{Q}$

Function ECP-FETCH$(i, t, \mathcal{Q}, \{\mathcal{L}_{i,j}\})$
- **foreach** $(r, \mathcal{K}^{(r)}, \{C_j^{(r)}\}) \in \mathcal{Q}$ $(t_{\text{last},i} < r \le t)$ **do**
  - Download $\{C_j^{(r)}\}_{j\in\mathcal{K}^{(r)}}$ into $\{\mathcal{L}_{i,j}\}$
- **return** $\{\mathcal{L}_{i,j}\}$

Function ECP-SYNC$(i, t, \mathbf{w}_i, \{\mathcal{L}_{i,j}\})$
- **if** $t - t_{\text{last},i} \le 1$ **then return** $\mathbf{w}_i$
- Download unprefetched packs in $[t_{\text{last},i} + 1, t - 1]$
- Initialize prefetched update $\tilde{\mathbf{g}}_{\text{pre}} \leftarrow \mathbf{0}$
- **foreach** $j \in [J]$ *such that* $\mathcal{L}_{i,j} \neq \emptyset$ **do**
  - Compute $A_{\text{pre},j}$ and $C_{\text{pre},j}$ as Eq. (20)
  - Decrypt $\text{pt}_{\text{pre},j}$ as Eq. (21)
  - Decode $\tilde{\mathbf{g}}_{\text{pre},j} \leftarrow \text{Dcd}(\text{pt}_{\text{pre},j}, \Delta)$
- Clear $\{\mathcal{L}_{i,j}\} \leftarrow \emptyset$ and **return** $\mathbf{w}_i + \tilde{\mathbf{g}}_{\text{pre}}$

Consequently, when the server sums $\text{ct}_{i,j}^{(t)} + \delta_{i,j}^{(t)}$ over $i \in \widehat{\mathcal{S}}^{(t)}$ and exchanges the double summation over $i \in \widehat{\mathcal{S}}^{(t)}$ and $k \notin \widehat{\mathcal{S}}^{(t)}$, the exact aggregate key contribution $a_j^{(t)} S$ is restored:

$$
\begin{aligned}
C_j^{(t)} &= \left[\sum_{i\in\widehat{\mathcal{S}}^{(t)}} \left(\text{ct}_{i,j}^{(t)} + \delta_{i,j}^{(t)}\right)\right]_q \\
&= \left[a_j^{(t)} \sum_{i\in\widehat{\mathcal{S}}^{(t)}} \left(\tilde{s}_{i,j}^{(t)} + \kappa_{i,j}^{(t)}\right) + \tilde{p}\widehat{E}_j^{(t)} + \text{pt}_j^{(t)}\right]_q \\
&= \Bigg[a_j^{(t)}\Big(\sum_{i\in\widehat{\mathcal{S}}^{(t)}} s_i + \sum_{k\notin\widehat{\mathcal{S}}^{(t)}} \underbrace{\sum_{i\in\widehat{\mathcal{S}}^{(t)}} \widehat{\lambda}_i^{(t)} s_{k,i}}_{=s_k} + b_j^{(t)} \underbrace{\sum_{i\in\widehat{\mathcal{S}}^{(t)}} \widehat{\lambda}_i^{(t)} z_i}_{=0}\Big) \\
&\qquad + \tilde{p}\widehat{E}_j^{(t)} + \text{pt}_j^{(t)}\Bigg]_q = \left[a_j^{(t)} S + \tilde{p}\widehat{E}_j^{(t)} + \text{pt}_j^{(t)}\right]_q,
\end{aligned}
\tag{19}
$$

where $\widehat{E}_j^{(t)} = \sum_{i\in\widehat{\mathcal{S}}^{(t)}} (e_{i,j}^{(t)} + \hat{e}_{i,j}^{(t)})$ and $\text{pt}_j^{(t)} = \sum_{i\in\widehat{\mathcal{S}}^{(t)}} \text{pt}_{i,j}^{(t)}$. Thus, the decoded pack update satisfies $\tilde{\mathbf{g}}_j^{(t)} = \text{Dcd}(\text{pt}_j^{(t)}, \Delta) \approx \sum_{i\in\widehat{\mathcal{S}}^{(t)}} \omega_i^{(t)} \mathbf{x}_{i,j}^{(t)}$ under the fixed aggregation weights $\omega_i^{(t)}$, without requiring plaintext re-encoding or post-decryption rescaling.

D. ECP: Ciphertext Prefetching

The ECP protocol resolves the **Sparsity and Prefetching Paradox (Q3)** by unifying the sparse update unit with the prefetching unit at the pack level. Concretely, each pack corresponds to one CKKS ciphertext that packs $n/2 = 4096$ update scalars into a single ring element.

1) Sparse Ciphertext Prefetching: The ECP module prefetches sparse, pack-level aggregated ciphertexts during idle rounds to synchronize intermittently available clients before they rejoin training. Its core idea is *pack-level sparse prefetching driven by the consensus pack mask* $\mathcal{K}^{(r)}$: instead of downloading all J packs synchronously, a rejoining client fetches only the selected consensus packs $\{C_j^{(r)}\}_{j\in\mathcal{K}^{(r)}}$, accumulates them homomorphically over $\mathcal{R}_q$, and decrypts each accumulated pack once upon resuming training. As illustrated in Fig. 4, Alg. 3 formalizes the ECP Downstream Module in three phases following the adaptive scheduling paradigm of FedFetch [20]:

Prepare Phase. At round t, the server selects the confirmed active set $\mathcal{S}^{(t)} \subseteq \mathcal{P}^{(t)}$, the unselected inactive set $\bar{\mathcal{S}}^{(t)} = \mathcal{C} \setminus \mathcal{S}^{(t)}$, and a presampled candidate pool $\mathcal{P}^{(t^*)} \subseteq \mathcal{C}$ of size $|\mathcal{P}^{(t^*)}| = \lceil \beta K \rceil$ targeting a future round $t^* = t + R$, where R is the prefetch window size and $\beta \ge 1$ is the over-commitment factor mitigating pre-round stragglers, satisfying $\widehat{\mathcal{S}}^{(t)} \subseteq \mathcal{S}^{(t)} \subseteq \mathcal{P}^{(t)} \subseteq \mathcal{C}$. Each presampled client $i \in \mathcal{P}^{(t^*)}$ is scheduled to start prefetching at round $t_{\text{start},i}^{(t^*)} \in \{t, \dots, t^*-1\}$ based on its downlink bandwidth and the exponential moving average (EMA) round duration estimate $\hat{d}^{(t)} = \rho d^{(t-1)} + (1-\rho)\hat{d}^{(t-1)}$. Meanwhile, the server invokes ECP-QUEUE to maintain a sliding queue $\mathcal{Q}$ of the most recent R rounds' sparse aggregates $\{C_j^{(r)}\}_{j\in\mathcal{K}^{(r)}}$, acting as a pure ciphertext accumulator without maintaining an encrypted full model.

Prefetch Phase. At round t, each presampled client $i \in \mathcal{P}^{(t^*)}$ whose scheduled start round satisfies $t_{\text{start},i}^{(t^*)} \le t < t^*$ invokes ECP-FETCH to scan the queue $\mathcal{Q}$. An aggregated ciphertext $C_j^{(r)}$ is downloaded only if three conditions hold: (i) client i missed round r $(t_{\text{last},i} < r)$; (ii) round r has already been aggregated $(r \le t)$; and (iii) pack j belongs to the consensus pack mask $(j \in \mathcal{K}^{(r)})$. The downloaded ciphertexts are stored in the local pack buffer $\mathcal{L}_{i,j}$, indexed by r to avoid duplicate downloads; any remaining packs are deferred to subsequent idle rounds.

Train Phase. At the start of round t, each active client $i \in \mathcal{S}^{(t)}$ that has been absent for more than one round $(\ell_i^{(t)} = t - t_{\text{last},i} > 1)$ invokes ECP-SYNC: it first fetches any remaining unprefetched packs from the missed rounds $r \in \{t_{\text{last},i} + 1, \dots, t - 1\}$ (shown as "Ciphertext Fetch" in Fig. 4), homomorphically merges its cached ciphertexts in $\mathcal{L}_{i,j}$ into the cross-round accumulated ciphertext $C_{\text{pre},j}$, and synchronizes its local model in one local decryption per pack via $\mathbf{w}_i \leftarrow \mathbf{w}_i + \tilde{\mathbf{g}}_{\text{pre}}$. Prefetching is advantageous when $\sum_{r=t_{\text{last},i}+1}^{t-1} |\mathcal{K}^{(r)}| < J$, as it replaces a full-model download with sparse pack retrievals. However, if the absence span exceeds the queue capacity $(\ell_i^{(t)} > R)$ or the total missed packs would exceed J, the server accumulates the missed updates per pack $j \in [J]$ in $\mathcal{R}_q$ so client i downloads at most J composite pack ciphertexts.

2) Client One-Step Decryption: ECPFL avoids per-round decryptions by letting clients accumulate missed updates into a single cross-round accumulated ciphertext $C_{\mathrm{pre},j}$ per pack. By Eq. (16) and Eq. (19), every completed round r yields an aggregated ciphertext $C_j^{(r)} = [a_j^{(r)} S + \tilde{p} E_j^{(r)} + \mathrm{pt}_j^{(r)}]_q$ (with $E_j^{(r)} = \widehat{E}_j^{(r)}$ if $\mathcal{D}^{(r)} \neq \emptyset$) encrypted under the identical aggregate decryption key S. Because Shamir key-contribution restoration preserves this exact algebraic structure across rounds regardless of client dropouts, client i can homomorphically sum the cached ciphertexts in $\mathcal{L}_{i,j}$. Let $\mathcal{H}_{i,j} = \{r \mid (r, C_j^{(r)}) \in \mathcal{L}_{i,j}\}$ denote the set of missed rounds in which pack j was selected. Substituting $C_j^{(r)}$ and factoring out the round-invariant key S yields the cross-round accumulated ciphertext $C_{\mathrm{pre},j}$ step by step:

$$
\begin{aligned}
C_{\mathrm{pre},j} &= \mathrm{HE.Eval}\Big(\big\{C_j^{(r)}\big\}_{r\in\mathcal{H}_{i,j}}\Big) = \Bigg[\sum_{r\in\mathcal{H}_{i,j}} C_j^{(r)}\Bigg]_q \\
&= \Bigg[\sum_{r\in\mathcal{H}_{i,j}} \Big(a_j^{(r)}\cdot S + \tilde{p}\cdot E_j^{(r)} + \mathrm{pt}_j^{(r)}\Big)\Bigg]_q \\
&= \Bigg[\underbrace{\Bigg(\sum_{r\in\mathcal{H}_{i,j}} a_j^{(r)}\Bigg)}_{A_{\mathrm{pre},j}}\cdot S + \tilde{p}\underbrace{\sum_{r\in\mathcal{H}_{i,j}} E_j^{(r)}}_{E_{\mathrm{pre},j}} + \underbrace{\sum_{r\in\mathcal{H}_{i,j}} \mathrm{pt}_j^{(r)}}_{\mathrm{pt}_{\mathrm{pre},j}}\Bigg]_q \\
&= \Big[\, A_{\mathrm{pre},j}\cdot S + \tilde{p}\cdot E_{\mathrm{pre},j} + \mathrm{pt}_{\mathrm{pre},j}\,\Big]_q, \qquad (20)
\end{aligned}
$$

where $A_{\mathrm{pre},j} = \big[\sum_{r\in\mathcal{H}_{i,j}} a_j^{(r)}\big]_q$ is the accumulated public polynomial, $E_{\mathrm{pre},j} = \sum_{r\in\mathcal{H}_{i,j}} E_j^{(r)}$ is the accumulated noise polynomial, and $\mathrm{pt}_{\mathrm{pre},j} = \sum_{r\in\mathcal{H}_{i,j}} \mathrm{pt}_j^{(r)}$ is the accumulated plaintext polynomial. Notably, $C_{\mathrm{pre},j}$ has the same single-ring-element size as a single-round ciphertext regardless of $|\mathcal{H}_{i,j}|$.

Unlike traditional multi-key decryption protocols that require interactive decryption-share uploads, ECPFL performs only one local decryption per pack. Using (S, σ_a), the client computes $A_{\mathrm{pre},j} = \big[\sum_{r\in\mathcal{H}_{i,j}} \mathrm{HE.PRPG}(\sigma_a, r \parallel j)\big]_q$ and extracts $\mathrm{pt}_{\mathrm{pre},j}$ via HE.Dec:

$$
\begin{aligned}
\mathrm{pt}_{\mathrm{pre},j} &= \mathrm{HE.Dec}\big(A_{\mathrm{pre},j}, S, C_{\mathrm{pre},j}\big) = \Big[\big[C_{\mathrm{pre},j} - A_{\mathrm{pre},j}S\big]_q\Big]_p \\
&= \Big[\big(A_{\mathrm{pre},j}S + \tilde{p}E_{\mathrm{pre},j} + \mathrm{pt}_{\mathrm{pre},j}\big) - A_{\mathrm{pre},j}S\Big]_{q,p} \\
&= \Big[\big[\,\tilde{p}\cdot E_{\mathrm{pre},j} + \mathrm{pt}_{\mathrm{pre},j}\,\big]_q\Big]_p = \mathrm{pt}_{\mathrm{pre},j}, \qquad (21)
\end{aligned}
$$

where subtracting $A_{\mathrm{pre},j}\cdot S$ cancels all accumulated public-key masks in a single step and reduction modulo p removes $\tilde{p}\cdot E_{\mathrm{pre},j}$ whenever $\|\tilde{p}\cdot E_{\mathrm{pre},j} + \mathrm{pt}_{\mathrm{pre},j}\|_\infty < q/2$. By the linearity of $\mathrm{Dcd}(\cdot,\Delta)$, the client decodes $\tilde{\mathbf{g}}_{\mathrm{pre},j} = \mathrm{Dcd}(\mathrm{pt}_{\mathrm{pre},j},\Delta) = \sum_{r\in\mathcal{H}_{i,j}} \tilde{\mathbf{g}}_j^{(r)}$ in one step, assembles the prefetched update $\tilde{\mathbf{g}}_{\mathrm{pre}}$, and updates $\mathbf{w}_i \leftarrow \mathbf{w}_i + \tilde{\mathbf{g}}_{\mathrm{pre}}$.

V. Security Analysis

In this section, we analyze the security and correctness of ECPFL against semi-honest entities and across its core protocol mechanisms.

A. Entity-Level Security Guarantees

1) Resilience Against Semi-Honest Clients:

Theorem 1 (Individual Ciphertext Confidentiality)**.** *Under the dRLWE assumption and (τ, N)-Shamir secret sharing, for any honest client $h \in \mathcal{H}^{(t)}$, the uploaded ciphertext $\mathrm{ct}_{h,j}^{(t)}$ is computationally indistinguishable from uniform over $\mathcal{R}_q$ against any coalition of $\kappa \leq \tau - 2$ semi-honest clients.*

Proof. Client h encrypts $\mathrm{pt}_{h,j}^{(t)}$ under $\tilde{s}_{h,j}^{(t)} = [s_h + \lambda_h^{(t)} \sum_{k\in\bar{\mathcal{S}}^{(t)}} s_{k,h} + \lambda_h^{(t)} b_j^{(t)} z_h]_q$ in Eq. (14). Although other clients hold (S, σ_a, σ_b), subtracting $a_j^{(t)} S$ from $\mathrm{ct}_{h,j}^{(t)}$ leaves the residual mask $a_j^{(t)}(\tilde{s}_{h,j}^{(t)} - S)$. Because $\kappa \leq \tau - 2 < \tau$, the degree-$(\tau-1)$ polynomials $F_h(x)$ and $\Psi(x)$ remain undetermined over $\mathcal{R}_q$, keeping s_h and z_h secret. Moreover, distinct per-pack polynomials $a_j^{(t)}$ and $b_j^{(t)}$ prevent cross-pack and cross-round linear elimination of s_h and z_h [44]. By the dRLWE assumption, $a_j^{(t)}\tilde{s}_{h,j}^{(t)} + \tilde{p}e_{h,j}^{(t)}$ is indistinguishable from uniform over $\mathcal{R}_q$, hiding $\mathrm{pt}_{h,j}^{(t)}$. □

2) Resilience Against Semi-Honest Server:

Theorem 2 (Server Obliviousness)**.** *Without client–server collusion, the server $\mathcal{A}_{\mathrm{srv}}$ cannot obtain any individual update $\mathrm{pt}_{i,j}^{(t)}$ or the aggregated update $\mathrm{pt}_j^{(t)}$.*

Proof. The server $\mathcal{A}_{\mathrm{srv}}$ observes $\mathrm{ct}_{i,j}^{(t)} = [a_j^{(t)}\tilde{s}_{i,j}^{(t)} + \tilde{p}e_{i,j}^{(t)} + \mathrm{pt}_{i,j}^{(t)}]_q$ and computes $C_j^{(t)} = [a_j^{(t)}S + \tilde{p}E_j^{(t)} + \mathrm{pt}_j^{(t)}]_q$, but never acquires $\{s_i\}$, S, $\{s_{k,i}, z_i\}$, or (σ_a, σ_b). By PRPG security and the dRLWE assumption, the masking terms $a_j^{(t)}\tilde{s}_{i,j}^{(t)} + \tilde{p}e_{i,j}^{(t)}$ and $a_j^{(t)}S + \tilde{p}E_j^{(t)}$ are computationally indistinguishable from random elements in $\mathcal{R}_q$. Without colluding with any client holding (S, σ_a), $\mathcal{A}_{\mathrm{srv}}$ cannot execute HE.Dec and learns neither $\mathrm{pt}_{i,j}^{(t)}$ nor $\mathrm{pt}_j^{(t)}$. □

3) Resilience Against Client Collusion:

Theorem 3 (Collusion Resistance)**.** *Against at most $\kappa \leq \tau - 2$ colluding clients $\mathcal{K}_{\mathrm{cor}} \subset \mathcal{C}$ in any valid round t ($|\widehat{\mathcal{S}}^{(t)}| \geq \tau$), no honest client's key s_h, zero-share z_h, or individual update $\mathrm{pt}_{h,j}^{(t)}$ ($h \in \mathcal{H}^{(t)} = \widehat{\mathcal{S}}^{(t)} \setminus \mathcal{K}_{\mathrm{cor}}$) is recoverable; the coalition learns at most $\sum_{h\in\mathcal{H}^{(t)}} \mathrm{pt}_{h,j}^{(t)}$.*

Proof. For each honest client $h \in \mathcal{H}^{(t)}$, the coalition holds $\kappa \leq \tau - 2$ shares of $F_h(x)$ and at most $(\tau-2)+1 = \tau - 1 < \tau$ evaluations of $\Psi(x)$ (including $\Psi(0) = 0$), leaving $s_h = F_h(0)$ and $z_h = \Psi(\alpha_h)$ information-theoretically undetermined over $\mathcal{R}_q$. Thus, individual ciphertexts $\mathrm{ct}_{h,j}^{(t)}$ cannot be decrypted (Theorem 1). Decrypting $C_j^{(t)}$ via (S, σ_a) and subtracting the coalition's known updates leaves the honest sum $\sum_{h\in\mathcal{H}^{(t)}} \mathrm{pt}_{h,j}^{(t)}$. Since $|\mathcal{H}^{(t)}| \geq \tau - (\tau-2) = 2$, this single equation in at least two unknowns cannot isolate any individual $\mathrm{pt}_{h,j}^{(t)}$. □

B. Protocol Correctness and Security

1) Vote Correctness and Confidentiality:

Lemma 1 (Vote Correctness and Confidentiality)**.** *When $|\mathcal{S}^{(t)}| \geq \tau$, $\kappa \leq \tau - 2$, $|\mathcal{S}^{(t)}| < p$, and $\|\tilde{p}E_0^{(t)} +$*

$\sum_{i\in\mathcal{S}^{(t)}} v_i^{(t)}(X)\|_\infty < q/2$, *server aggregation in Eq.* (13) *recovers the exact pack tally* $\nu_j^{(t)} = \sum_{i\in\mathcal{S}^{(t)}} v_{i,j}^{(t)}$ *while keeping each honest vote bitmap* $\mathbf{v}_h^{(t)}$ *confidential.*

Proof. Since $|\mathcal{S}^{(t)}| \geq \tau > \deg(\Psi)$, Eq. (9) gives $\left[\sum_{i\in\mathcal{S}^{(t)}} \lambda_i^{(t)} z_i\right]_q = 0$, so summing $\hat{v}_i^{(t)}$ in Eq. (13) and reducing modulo q and p removes $\tilde{p}E_0^{(t)}$ without wrap-around and recovers $V^{(t)}(X) = \sum_{j=1}^{J} \nu_j^{(t)} X^{j-1}$. For confidentiality, $a_0^{(t)}\lambda_h^{(t)} z_h + \tilde{p}e_{h,0}^{(t)}$ masks $v_h^{(t)}(X)$ against $\mathcal{A}_{\text{srv}}$ under dRLWE, while z_h is unknown to $\mathcal{A}_{\text{cli}}$ (Theorem 3). Moreover, cross-round elimination $D = [A_2\hat{v}_h^{(1)} - A_1\hat{v}_h^{(2)}]_q$ with $A_t = [a_0^{(t)}\lambda_h^{(t)}]_q$ causes $\tilde{p}(A_2 e_{h,0}^{(1)} - A_1 e_{h,0}^{(2)})$ to wrap around modulo q; since $\gcd(p,q) = 1$, the unknown wrap-around residue modulo p conceals $\mathbf{v}_h^{(t)}$. □

2) Dropout Compensation Security:

Theorem 4 (Dropout Compensation Security)**.** *For any surviving set* $\widehat{\mathcal{S}}^{(t)} \subseteq \mathcal{S}^{(t)}$ *with* $|\widehat{\mathcal{S}}^{(t)}| \geq \tau$*, aggregation in Eq.* (16) *and Eq.* (19) *restores the exact key contribution* $a_j^{(t)} S$*, and the compensation ciphertext* $\delta_{h,j}^{(t)}$ *is computationally indistinguishable from uniform over* $\mathcal{R}_q$ *against* $\mathcal{A}_{\text{srv}}$ *and* $\mathcal{A}_{\text{cli}}$ *(*$\kappa \leq \tau - 2$*).*

Proof. When $\mathcal{D}^{(t)} = \emptyset$, Eq. (9) over $|\mathcal{S}^{(t)}| \geq \tau$ gives $\sum_{i\in\mathcal{S}^{(t)}} \tilde{s}_{i,j}^{(t)} \equiv S \pmod q$. When $\mathcal{D}^{(t)} \neq \emptyset$, adding $\kappa_{i,j}^{(t)}$ (Eq. (17)) cancels the stale $\lambda_i^{(t)}$ terms in Eq. (18), and summing over $|\widehat{\mathcal{S}}^{(t)}| \geq \tau$ reconstructs $\sum_{k\notin\widehat{\mathcal{S}}^{(t)}} s_k$ and cancels z_i, yielding $C_j^{(t)} = [a_j^{(t)} S + \tilde{p}\widehat{E}_j^{(t)} + \text{pt}_j^{(t)}]_q$ without revealing S to the server. Furthermore, $\delta_{h,j}^{(t)} = [a_j^{(t)}\kappa_{h,j}^{(t)} + \tilde{p}\hat{e}_{h,j}^{(t)}]_q$ masks $\{s_{k,h}\}$ with $(\widehat{\lambda}_h^{(t)} - \lambda_h^{(t)})b_j^{(t)} z_h$ and fresh noise $\hat{e}_{h,j}^{(t)} \leftarrow \chi_e$; since z_h is unknown (Theorem 3) and $b_j^{(t)}$ is fresh per pack and round, $\delta_{h,j}^{(t)}$ is indistinguishable from uniform over $\mathcal{R}_q$ under dRLWE. □

3) Cross-Round Prefetching Security:

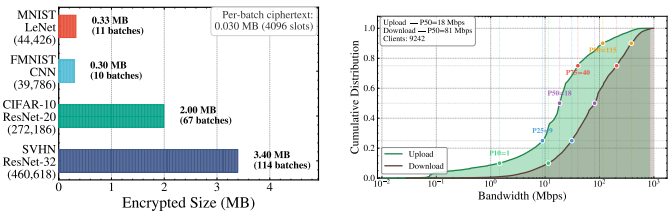
Theorem 5 (Cross-Round Prefetching Security)**.** *Whenever* $\|\tilde{p}E_{\text{pre},j} + \text{pt}_{\text{pre},j}\|_\infty < q/2$*, (i) one-step decryption in Eq.* (21) *recovers* $\text{pt}_{\text{pre},j} = \sum_{r\in\mathcal{H}_{i,j}} \text{pt}_j^{(r)}$*; and (ii)* $C_{\text{pre},j}$ *is computationally indistinguishable from uniform over* $\mathcal{R}_q$ *to* $\mathcal{A}_{\text{srv}}$ *and leaks no single-client update to* $\mathcal{A}_{\text{cli}}$*.*

Proof. (i) Summing $\{C_j^{(r)}\}_{r\in\mathcal{H}_{i,j}}$ in Eq. (20) gives $C_{\text{pre},j} = [A_{\text{pre},j}S + \tilde{p}E_{\text{pre},j} + \text{pt}_{\text{pre},j}]_q$. Subtracting $A_{\text{pre},j}S$ and reducing modulo q and p removes $\tilde{p}E_{\text{pre},j}$ without wrap-around, recovering $\text{pt}_{\text{pre},j}$ in one step. (ii) Without (S, σ_a), $A_{\text{pre},j}S + \tilde{p}E_{\text{pre},j}$ hides $\text{pt}_{\text{pre},j}$ from $\mathcal{A}_{\text{srv}}$ under dRLWE. For authorized clients, $C_{\text{pre},j}$ is a sum of public round aggregates $\{C_j^{(r)}\}_{r\in\mathcal{H}_{i,j}}$, revealing no additional information beyond $\{\text{pt}_j^{(r)}\}_{r\in\mathcal{H}_{i,j}}$, where individual updates remain protected by Theorem 3. □

VI. Performance Evaluation

A. Experimental Setup

1) Basic Configuration, Client Scalability, and Dropout Resilience: All experiments run on a workstation with an Intel Core i7-14700, 32 GB RAM, and an NVIDIA RTX 3050 GPU. We use PyTorch for local training and a compiled MKHE dynamic library for homomorphic encryption.



(a) Encrypted model sizes and CKKS batch partitions across datasets.

(b) Cumulative distribution of client upload and download bandwidth.

Fig. 5. **Dataset batch partitions and bandwidth profiles.** Model sizes and client bandwidth distributions across experimental datasets.

TABLE II Scalability Analysis of Accuracy, Communication Volume, and Client Key Storage Across Client Scales on CIFAR-10

Client Scale		**Training & Comm.** ($T = 50$)				**Client Setup**	
N	K	**AC (%)**	**TV** (GB)	**Base** (GB)	**TT** (s)	**Storage** $\mathcal{O}(N)$	**Setup** (s)
50	**25**	**82.77**	**13.71**	**52.15**	**962.5**	**3.33** MB	**0.42**
100	50	82.91	27.42	104.30	1894.0	6.44 MB	1.65
500	250	83.08	137.10	521.50	9410.5	31.44 MB	41.20
1000	500	83.15	274.20	1043.00	18790.0	62.69 MB	164.80

Note: Base = FedAvg+CKKS TV; Storage $\mathcal{O}(N)$ = per-client key and share storage.

2) Datasets and Models: Fig. 5(a) illustrates our evaluation of ECPFL across four diverse datasets [58], [59], [60] varying in data scales, model parameters, and SIMD batch sizes:

- **MNIST** (LeNet, 11 packs): 10-class handwritten digits; 60,000 training and 10,000 test samples.
- **Fashion-MNIST (FMNIST)** (CNN, 10 packs): 10-class grayscale clothing images; 60,000 training and 10,000 test samples.
- **CIFAR-10** (ResNet-20, 67 packs): 10-class color images; 50,000 training and 10,000 test samples.
- **SVHN** (ResNet-32, 114 packs): real-world RGB digits; 73,257 training and 26,032 test samples.

In addition, we use the bandwidth measurements of [61], collected from heterogeneous devices such as smartphones and PCs. The lowest upload and download bandwidths on this dataset are 0.03 Mbps and 0.01 Mbps, respectively. As shown in Fig. 5(b), the median download bandwidth is 81.29 Mbps, significantly higher than the median upload bandwidth of 18.42 Mbps.

3) Baselines for Comparison: To comprehensively evaluate the efficiency of ECPFL, we compare it with four representative state-of-the-art protocols:

- **FedAvg+CKKS**: a naive integration of single-key CKKS with standard FedAvg, encrypting updates with SIMD packing.
- **FedPHE** [11]: a single-key CKKS framework with pack-level Top-k sparsification for selective gradient aggregation.
- **ESAFL** [31]: a multi-key secure aggregation protocol that improves ciphertext efficiency via SIMD slot packing.
- **MASER** [40]: a multi-key HE framework that reduces computation and communication overhead through

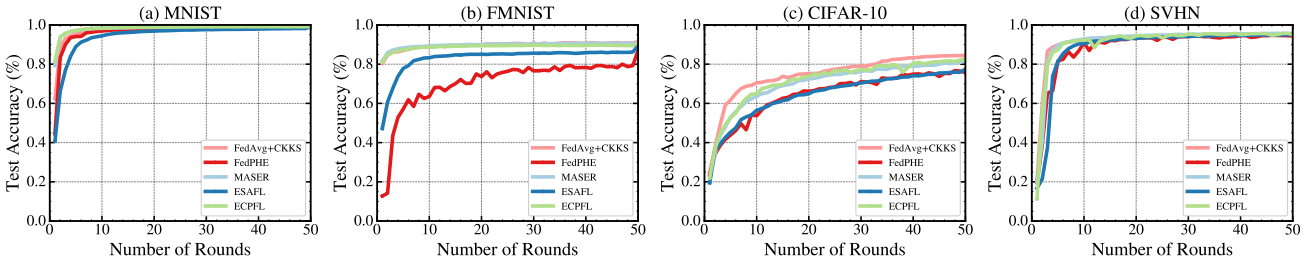


Fig. 6. **Test accuracy convergence trajectories.** Comparison across four benchmark datasets under the non-IID setting ($\alpha = 1.0, T = 50$ rounds).

TABLE III OVERALL COMPARISON OF TEST ACCURACY, TOTAL COMMUNICATION VOLUME, AND TRAINING TIME ACROSS FOUR BENCHMARK DATASETS

Method	**MNIST (LeNet)** ($\alpha = 1.0, \theta = 0.1, \zeta = 0.1$)			**FMNIST (CNN)** ($\alpha = 1.0, \theta = 0.3, \zeta = 0.1$)			**CIFAR-10 (ResNet-20)** ($\alpha = 1.0, \theta = 0.7, \zeta = 0.5$)			**SVHN (ResNet-32)** ($\alpha = 1.0, \theta = 0.7, \zeta = 0.5$)		
	AC (%)	**TV** (GB)	**TT** (s)	**AC** (%)	**TV** (GB)	**TT** (s)	**AC** (%)	**TV** (GB)	**TT** (s)	**AC** (%)	**TV** (GB)	**TT** (s)
FedAvg+CKKS	**99.23**	8.56	315.0	**91.46**	7.78	285.4	**84.41**	52.15	1756.3	**95.73**	89.60	3141.6
FedPHE (Top-ζ)	99.10	3.84	201.5	87.00	3.95	179.4	75.77	31.75	1221.5	94.44	54.38	2445.1
ESAFL	99.11	4.67	240.0	89.51	3.89	244.4	77.51	26.46	1402.6	95.54	45.14	2363.5
MASER	99.13	8.31	466.7	90.66	7.17	427.8	81.91	40.92	1927.9	95.63	71.02	3308.4
ECPFL-UN (Ours)	99.19	4.21	439.9	91.20	2.86	323.2	82.77	20.03	1564.9	95.11	34.89	986.5
ECPFL-TH-θ (Ours)	99.10	4.06	422.5	90.74	2.15	259.8	75.58	**7.60**	**689.2**	94.17	**19.10**	**577.6**
ECPFL-RK-θ (Ours)	99.15	**0.80**	**129.9**	90.14	**1.19**	**95.8**	79.24	18.66	1441.4	95.11	31.68	923.2

Note: Bold values indicate the best result in each column, and underlined values indicate the second-best result.

consensus-based model pruning and parameter slicing.

We implement FedAvg+CKKS and FedPHE with the TenSEAL library [62], MASER with the xMK-CKKS library plus partial-decryption upload, and ESAFL with a bit-packing scheme that packs two quantized parameters into one CKKS slot (50% size reduction on all datasets). We further benchmark three voting variants: ECPFL-UN retains every pack receiving at least one vote; ECPFL-RK-θ retains the top θ proportion of packs by global vote tally; and ECPFL-TH-θ retains packs voted by at least a θ fraction of the active clients.

4) Hyperparameters: All methods utilize the Adam optimizer with learning rate $\eta = 0.001$, a batch size of 64, and $\mathcal{E} = 3$ local epochs. The client sampling rate is 0.5, and the over-commitment parameter is $\beta = 1.3$ [20] to avoid stragglers. The default settings for ECPFL are as follows: pack-level sparsification is $\zeta = 0.1$ for MNIST and FMNIST, and $\zeta = 0.5$ for CIFAR-10 and SVHN. The prefetch window is set to $R = 3$, and dual-decay parameters are $\gamma_d = 0.3$ and $\gamma_s = 0.5$. EMA smoothing with $\rho = 0.125$ is used to predict the training time for the prefetch round. For all baselines, CKKS uses the same polynomial modulus degree $n = 8192$ and a coefficient modulus sequence of $\{60, 40, 40, 60\}$ bits.

As shown in Table II, we first investigate ECPFL under four client scales $N \in \{50, 100, 500, 1000\}$, with $K \in \{25, 50, 250, 500\}$ clients sampled each round. We observe that the superiority of ECPFL is already well manifested at $N = 50$, and thus adopt $N = 50$ ($K = 25, \tau = 13$) for the remaining comparative experiments.

5) Metrics: We evaluate our framework using *Test Accuracy* (AC) averaged over five independent runs. Communication overhead is quantified by *Upload Volume* (UV), *Prefetch Volume* (PV), *Downstream Fetch Volume* (DV), and *Total Volume* (TV = UV + DV). To separately characterize the physical downstream traffic introduced by prefetching, we additionally report *Physical Downstream Volume* (PDV = PV + DV). System latency is measured through *Upload Time* (UT), *Downstream Fetch Time* (DT), and the end-to-end *Total Training Time* (TT).

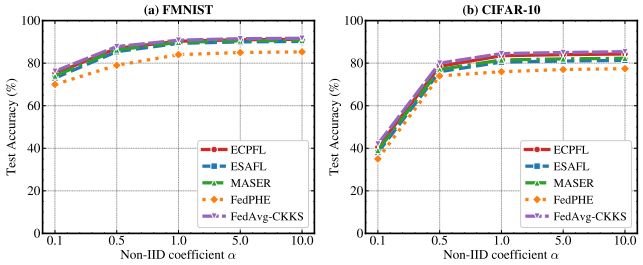


Fig. 7. **Test accuracy under varying non-IID degrees.** Comparison on FMNIST and CIFAR-10 across Dirichlet concentrations $\alpha \in \{0.1, 0.5, 1.0, 5.0, 10.0\}$.

B. Overall Performance

Fig. 6 and Table III summarize convergence, test accuracy, communication volume, and training time across the four datasets. ECPFL converges rapidly and stably, closely tracking the dense FedAvg+CKKS baseline. ECPFL-UN achieves 99.19%, 91.20%, 82.77%, and 95.11% on MNIST, FMNIST, CIFAR-10, and SVHN, respectively, remaining within 1.7 percentage points of the dense baseline while outperforming FedPHE by 4.20% on FMNIST and 7.00% on CIFAR-10.

ECPFL-RK achieves the lowest communication on MNIST and FMNIST, reducing TV by 10.7× and 6.5× and TT by 2.4× and 3.0× over FedAvg+CKKS. On the larger CIFAR-10

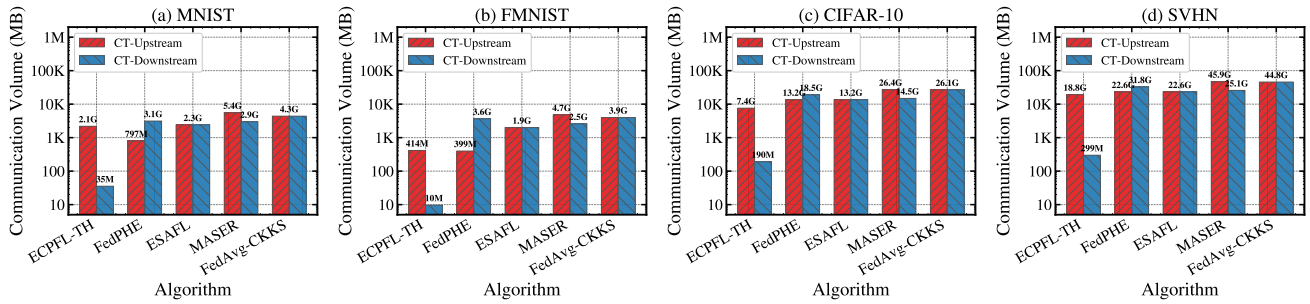


Fig. 8. **Cumulative communication volume.** Upstream and synchronous downstream ciphertext volume over 50 rounds across frameworks (log scale).

TABLE IV PER-ROUND BATCH COUNT, COMMUNICATION VOLUME, AND LATENCY BREAKDOWN ACROSS DATASETS

Dataset	Algorithm	Batches		Volume (MB)		Latency (s)	
		N_{up}	N_{agg}	UV	DV	UT	DT
MNIST	FedAvg+CKKS	275	11	87.7	87.7	269.1	182.6
	FedPHE	50	2	15.9	62.7	**48.9**	130.5
	ESAFL	150	6	47.8	47.8	146.8	99.6
	MASER	173	7	110.6	59.5	339.5	124.0
	ECPFL-RK	**50**	**2**	**15.9**	**0.3**	78.6	**2.3**
FMNIST	FedAvg+CKKS	250	10	79.7	79.7	244.6	166.0
	FedPHE	**25**	**1**	**8.0**	72.9	**24.5**	151.8
	ESAFL	125	5	39.9	39.9	122.3	83.0
	MASER	150	6	95.3	51.5	292.5	107.2
	ECPFL-RK	75	3	23.9	**0.4**	52.7	**0.8**
CIFAR-10	FedAvg+CKKS	1675	67	534.0	534.0	1638.7	1111.9
	FedPHE	850	34	271.0	379.2	831.6	789.6
	ESAFL	850	34	271.0	271.0	831.6	564.3
	MASER	849	34	541.3	296.8	1661.2	618.0
	ECPFL-TH	**476**	**19**	**151.8**	**3.8**	**598.0**	**15.0**
SVHN	FedAvg+CKKS	2875	114	916.6	916.6	2522.7	1137.7
	FedPHE	1450	58	462.3	651.3	1418.6	1356.3
	ESAFL	1450	58	462.3	462.3	1418.6	962.6
	MASER	1474	59	939.8	514.7	2884.1	1071.7
	ECPFL-TH	**1208**	**48**	**385.3**	**6.0**	**477.9**	**11.0**

Note: $N_{\text{up}}/N_{\text{agg}}$ = uploaded/aggregated batches; UV/DV and UT/DT are per-round.

TABLE V COMPONENT-WISE ABLATION STUDY OF THE ESP MODULE ON SVHN (RESNET-32, $\zeta = 0.5$, $\theta = 0.7$)

ESP Module Components					Performance Metrics		
LS	RF	PRD	CRD	RK	AC (%)	UV (MB)	ΔUV
–	–	–	–	–	**95.73**	45,429.7	Baseline
✓	–	–	–	–	95.07	41,553.9	-8.5%
✓	✓	–	–	–	95.01	41,798.3	-8.0%
✓	✓	✓	–	–	94.89	42,409.5	-6.6%
✓	✓	✓	✓	–	94.91	41,528.6	-8.6%
✓	✓	✓	✓	✓	94.90	**31,869.6**	$\mathbf{-29.8\%}$

and SVHN models, ECPFL-TH reduces TV by $6.9\times$ and $4.7\times$ over FedAvg+CKKS ($3.7\times$ over MASER) and accelerates TT by $2.5\times$ and $5.4\times$ over FedAvg+CKKS (and by $5.7\times$ over MASER on SVHN).

To evaluate statistical heterogeneity, we vary the Dirichlet parameter $\alpha \in \{0.1, 0.5, 1, 5, 10\}$ on FMNIST and CIFAR-10 in Fig. 7. ECPFL remains within roughly one percentage point of the dense FedAvg+CKKS upper bound across all α regimes while substantially saving bandwidth. Under severe heterogeneity ($\alpha = 0.1$), ECPFL reaches 75.1% on FMNIST and 40.5% on CIFAR-10, surpassing MASER, ESAFL, and FedPHE by up to 5.1% and 5.5%, respectively. At $\alpha = 10$, ECPFL reaches 91.2% on FMNIST and 84.3% on CIFAR-10, closely tracking FedAvg+CKKS (91.7% and 85.4%).

C. Overhead and Efficiency Evaluation

1) Communication Overhead: Fig. 8 reports the 50-round upstream and downstream volumes for ECPFL-TH and the baselines. ECPFL-TH lowers the synchronous downstream volume from 4.3 GB, 3.9 GB, 26.1 GB, and 44.8 GB under FedAvg+CKKS to 35 MB, 10 MB, 190 MB, and 299 MB on MNIST, FMNIST, CIFAR-10, and SVHN, corresponding to downstream reductions of $124\times$, $399\times$, $140\times$, and $153\times$. Relative to FedPHE, ECPFL-TH further lowers total communication by $1.8\times$, $4.2\times$, and $2.9\times$ on FMNIST, CIFAR-10, and SVHN. These gains stem from ESP's pack-level sparse consensus—which confines per-round aggregation to 2, 3, 19, and 48 ciphertext batches (Table IV)—and from ECP's asynchronous prefetching.

2) End-to-End Training Time: As shown in Table III and Table IV, ECPFL-RK completes MNIST, FMNIST, CIFAR-10, and SVHN in 129.9 s, 95.8 s, 1441.4 s, and 923.2 s ($2.4\times$, $3.0\times$, $1.2\times$, and $3.4\times$ faster than FedAvg+CKKS), while ECPFL-TH completes CIFAR-10 and SVHN in 689.2 s and 577.6 s ($2.5\times$ and $5.4\times$ speedups) by retaining only 19 and 48 aggregated batches. On the small 11-pack MNIST model, ECPFL-RK (2 packs) achieves a $2.4\times$ speedup, whereas ECPFL-UN and ECPFL-TH retain more packs whose voting round-trip offsets the modest absolute bandwidth savings.

D. Ablation Study

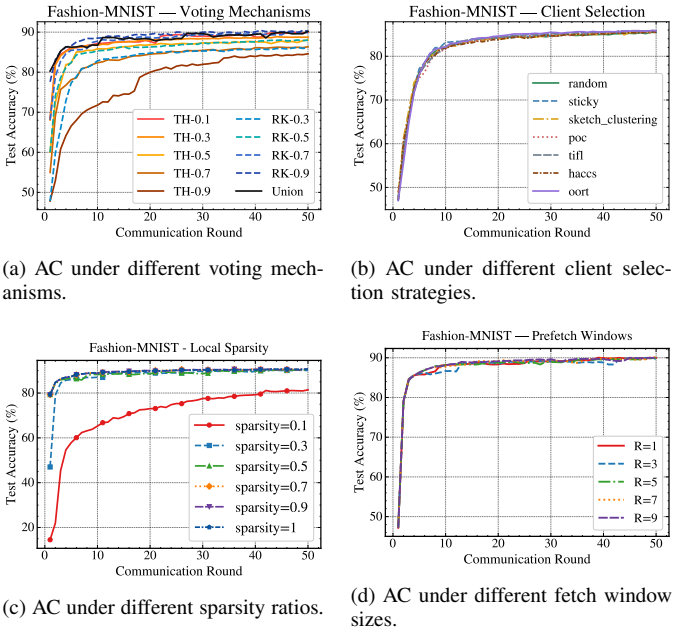
We conduct systematic ablation experiments on SVHN and CIFAR-10 to isolate the contribution of each ECPFL module.

1) Effectiveness of ESP: Table V isolates each ESP component on SVHN. Applying local sparsity (LS) alone with $\zeta = 0.5$ reduces UV by 8.5% (45429.7 to 41553.9 MB) while lowering accuracy marginally ($95.73\% \rightarrow 95.07\%$). Adding residual feedback (RF), per-round decay (PRD), and cross-round decay (CRD) stabilizes sparse aggregation, keeping accuracy within 0.82 percentage points of the dense baseline (94.91% vs. 95.73%). Rank voting (RK) at $\theta = 0.7$ then aligns the sparse selections across clients, avoiding ciphertext union expansion: UV drops by 23.3% over the non-RK stack and 29.8% over the dense baseline to 31869.6 MB at 94.90% accuracy.

TABLE VI REPRESENTATIVE CLIENT-LEVEL BANDWIDTH PROFILES AND DOWNSTREAM PREFETCH BREAKDOWN ON CIFAR-10 (RESNET-20, UNION, $R = 3$)

Client	Prt.	CTs	Bandwidth (Mbps)		Downstream (MB)			Offload
			DL	UL	PV	DV	PDV	PV/PDV
0	34	2,207	136.97	160.28	702.55	14.59	717.14	98.0%
1	29	1,889	177.35	14.59	702.33	**0.00**	702.33	**100.0%**
2	9	592	278.73	8.22	571.69	14.59	586.28	97.5%
3	1	67	67.15	2.18	702.77	**0.00**	702.77	**100.0%**
4	38	2,472	318.88	36.82	717.36	14.59	731.95	98.0%
5	36	2,350	307.66	150.35	704.35	**0.00**	704.35	**100.0%**
6	3	199	30.87	0.12	717.36	14.59	731.95	98.0%

Note: Prt. = participation rounds; CTs = uploaded ciphertexts; Offload = PV/PDV.



(a) AC under different voting mechanisms.

(b) AC under different client selection strategies.

(c) AC under different sparsity ratios.

(d) AC under different fetch window sizes.

Fig. 9. **Sensitivity analysis on FMNIST.** Test accuracy across voting variants, client selection strategies, local sparsity ratios, and prefetch windows.

2) Effectiveness of ECP: Table VI reports the per-client communication breakdown for CIFAR-10 with union voting and $R = 3$. With ECP, asynchronous prefetching (PV) accounts for 97.5% to 100.0% of each client's total downstream volume, while synchronous fetch volume (DV) drops to 0–14.59 MB. Clients 1, 3, and 5 achieve DV $= 0.00$ MB, completing synchronization entirely in the background, while frequently sampled clients (Clients 0 and 4) incur only 2.0% synchronous downstream traffic.

E. Sensitivity Analysis

To comprehensively evaluate the robustness of ECPFL, we conduct a sensitivity analysis on the FMNIST dataset across voting mechanisms, client selection strategies, local sparsity ratios, and prefetch window sizes in Fig. 9.

1) Impact of Voting Mechanisms: Fig. 9(a) compares the consensus variants. Union and rank-based variants (RK-0.3, RK-0.5) achieve the highest accuracy, converging to approximately 90% within 20 rounds, whereas TH-0.9 plateaus at 84% because an absolute 90% consensus threshold prematurely discards class-specific gradient features.

2) Impact of Client Selection Strategies: Fig. 9(b) evaluates seven client selection strategies: uniform Random [56], Sticky sampling [15], [20], LSH-based Sketch-Clustering [11], Power-of-Choice (PoC) [10], tier-based selection (TiFL) [12], clustered selection (HACCS) [13], and utility-guided selection (Oort) [14]. The accuracy trajectories of all seven strategies nearly coincide, converging to approximately 86% under the evaluated non-IID configuration: because ECP prefetches global ciphertexts during idle periods and fetches only minimal residual data upon selection, historically inactive stragglers contribute immediately without stalling synchronous aggregation.

3) Impact of Local Sparsity Ratio: Fig. 9(c) varies the local sparsity ratio $\zeta \in \{0.1, 0.3, 0.5, 0.7, 0.9, 1.0\}$, where $\zeta = 1.0$ corresponds to the dense baseline. For $\zeta \geq 0.3$, the trajectory closely matches the baseline and stabilizes near 90%, whereas $\zeta = 0.1$ degrades accuracy to approximately 80%, as excessive local pruning eliminates task-relevant information before consensus.

4) Impact of Prefetch Window Size R: Fig. 9(d) shows that trajectories for $R \leq 9$ are nearly indistinguishable (peak deviation below 0.3 percentage points around 90% test accuracy), since asynchronous prefetching decouples transmission from computation and prevents critical staleness across practical prefetch horizons.

VII. CONCLUSION AND FUTURE WORK

This paper proposes ECPFL, a dropout-resilient ciphertext prefetching FL framework that extends one-step aggregate decryption to client subsampling and upload-phase client dropout scenarios without peer-to-peer client communication or decryption-share uploads. The ESP protocol addresses the alignment and dropout challenges in encrypted sparsification by integrating a Shamir zero-sharing-based secure voting mechanism with a dropout-resilient pack-level sparse ciphertext upload and key-compensation mechanism, enabling aligned upstream compression and exact aggregate key-contribution restoration under fixed active-set weights. Building upon ESP, the ECP protocol allows intermittently available clients with limited bandwidth to asynchronously prefetch and accumulate sparse ciphertext updates during idle periods and recover multiple rounds of global updates through a single local decryption operation per pack, shifting most downstream communication out of the synchronous training phase.

Although our incremental KGC initialization eliminates online KGC involvement and persistent share-matrix storage, its quadratic one-time setup communication and linear per-client share storage remain scalability limitations. Future work will explore fully decentralized distributed key generation (DKG) and committee-based secret sharing to eliminate the quadratic setup overhead, and will investigate adaptive pack-level voting based on gradient consensus and client availability.

REFERENCES

[1] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečný, S. Mazzocchi, H. B. McMahan *et al.*, "Towards federated learning at scale: System design," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 1, 2019, pp. 374–388.

[2] L. Zhang, J. Xu, P. Vijayakumar, P. K. Sharma, and U. Ghosh, "Homomorphic encryption-based privacy-preserving federated learning in IoT-enabled healthcare system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2864–2880, 2022.

[3] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 2, 2020, pp. 429–450.

[4] R. Rahman, S. Shaon, and D. C. Nguyen, "Tackling heterogeneity in quantum federated learning: An integrated sporadic-personalized approach," *IEEE Trans. Comput.*, 2026.

[5] P. Ye, Y. Li, K. He, Q. Li, T. Qin, X. Wang, K. Yang, C. Zhang, and J. Chen, "Breaking the illusion: A critical study of backdoor defense in federated learning with non-IID data," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 13 264–13 279, 2025.

[6] K. He, H. Bai, Y. Li, J. Chen, and R. Du, "ACE-pFL: Accurate, efficient personalized federated learning with knowledge distillation," *IEEE/ACM Trans. Netw.*, vol. 33, no. 3, pp. 1265–1278, 2025.

[7] R. Wang, Y. Li, C. Zhang, and W. Wang, "Heterogeneous federated learning frameworks for balancing job completion time and model accuracy," *IEEE/ACM Trans. Netw.*, vol. 34, no. 1, pp. 123–135, 2026.

[8] P. Kairouz, H. B. McMahan *et al.*, "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, 2021.

[9] J. Wu and W. Zhang, "On the security of verifiable and oblivious secure aggregation for privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 4324–4326, 2024.

[10] Y. J. Cho, J. Wang, and G. Joshi, "Towards understanding biased client selection in federated learning," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2022, pp. 10 351–10 375.

[11] Q. Li, Z. Wen, Z. Wu, S. Huang, N. Wang, Y. Li, X. Liu, and B. He, "FedPHE: A secure and efficient federated learning system based on partial homomorphic encryption," *IEEE Trans. Dependable Secure Comput.*, 2025.

[12] Z. Chai, A. Ali, S. Zawad, S. Truex, A. Anwar, N. Baracaldo, Y. Zhou, H. Ludwig, F. Yan, and Y. Cheng, "TiFL: A tier-based federated learning system," in *Proc. IEEE Int. Symp. High-Perform. Parallel Distrib. Comput. (HPDC)*, 2020, pp. 125–136.

[13] J. Wolfrath, N. Sreekumar, D. Kumar, Y. Wang, and A. Chandra, "HACCS: Heterogeneity-aware clustered client selection for accelerated federated learning," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, 2022, pp. 985–995.

[14] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Proc. USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2021, pp. 19–35.

[15] S. He, Q. Yan, F. Wu, L. Wang, M. Lécuyer, and I. Beschastnikh, "GlueFL: Reconciling client sampling and model masking for bandwidth efficient federated learning," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 5, 2023, pp. 695–707.

[16] Z. Liu, J. Guo, W. Yang, J. Fan, K.-Y. Lam, and J. Zhao, "Dynamic user clustering for efficient and privacy-preserving federated learning," *IEEE Trans. Dependable Secure Comput.*, 2024.

[17] S. Xu, Y. Zheng, and Z. Hua, "Harnessing sparsification in federated learning: A secure, efficient, and differentially private realization," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2025, pp. 2354–2368.

[18] C. Dong, J. Weng, M. Li, J.-N. Liu, Z. Liu, Y. Cheng, and S. Yu, "Privacy-preserving and Byzantine-robust federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 889–904, 2023.

[19] D. Rothchild, A. Panda, E. Ullah, N. Ivkin, I. Stoica, V. Braverman, J. Gonzalez, and R. Arora, "FetchSGD: Communication-efficient federated learning with sketching," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 8253–8265.

[20] Q. Yan, A. Liu, S. He, M. Lécuyer, and I. Beschastnikh, "FedFetch: Faster federated learning with adaptive downstream prefetching," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, 2025, pp. 1–10.

[21] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 14 774–14 784.

[22] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients—how easy is it to break privacy in federated learning?" in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 16 937–16 947.

[23] B. Zhao, K. R. Mopuri, and H. Bilen, "iDLG: Improved deep leakage from gradients," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2020.

[24] P. Guo, R. Wang, S. Zeng, J. Zhu, H. Jiang, Y. Wang, Y. Zhou, F. Wang, H. Xiong, and L. Qu, "Exploring the vulnerabilities of federated learning: A deep dive into gradient inversion attacks," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2025.

[25] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[26] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1175–1191.

[27] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. ACM Symp. Theory Comput. (STOC)*, 2009, pp. 169–178.

[28] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur. (ASIACRYPT)*, 2017, pp. 409–437.

[29] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Techn. (EUROCRYPT)*, 2010, pp. 1–23.

[30] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Techn. (EUROCRYPT)*, 2018, pp. 360–384.

[31] J. Wu, W. Zhang, and F. Luo, "ESAFL: Efficient secure additively homomorphic encryption for cross-silo federated learning," *IEEE Trans. Dependable Secure Comput.*, 2025.

[32] Y. Cai, W. Ding, Y. Xiao, Z. Yan, X. Liu, and Z. Wan, "SecFed: A secure and efficient federated learning based on multi-key homomorphic encryption," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3817–3833, 2023.

[33] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Sparse binary compression: Towards distributed deep learning with minimal communication," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2019, pp. 1–8.

[34] ——, "Robust and communication-efficient federated learning from non-IID data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, 2019.

[35] X. Yuan *et al.*, "Robust and privacy-preserving federated learning with dropout-resilient homomorphic encryption," *Adv. Eng. Inform.*, 2026.

[36] Y. Pan, J. Yu, X. Yuan, C. Wang, and B. Li, "FedSHE: Efficient secure federated learning via segmental homomorphic encryption," *IEEE Trans. Dependable Secure Comput.*, 2024.

[37] H. Chen, W. Dai, M. Kim, and Y. Song, "Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 395–412.

[38] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Proc. Annu. Cryptol. Conf. (CRYPTO)*, 2012, pp. 643–662.

[39] J. Wu, T. Sun, F. Luo, H. Wang, and W. Zhang, "Secure multi-key homomorphic encryption with application to privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Security*, 2025.

[40] A. Al Omar, X. Yang, E. Choo, and O. Ardakanian, "MASER: Efficient privacy-preserving cross-silo federated learning with multi-key homomorphic encryption," in *Proc. IEEE Int. Conf. Big Data (BigData)*, 2025, pp. 3472–3481.

[41] C. Mouchet, J. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "An efficient threshold access-structure for RLWE-based multiparty homomorphic encryption," *J. Cryptol.*, vol. 36, no. 2, pp. 1–38, 2023.

[42] Y. Chang, Q. Li *et al.*, "Arbitrary-threshold fully homomorphic encryption with lower complexity," in *Proc. USENIX Secur. Symp.*, 2025.

[43] Y. Wang, F. Inoue, Y. Gu *et al.*, "Privacy-preserving federated learning via secret sharing and multi-key homomorphic encryption," *Inf. Sci.*, vol. 735, p. 123029, 2026.

[44] F. Colin de Verdière, A. Passelègue, and D. Stehlé, "On threshold fully homomorphic encryption with synchronized decryptors," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Techn. (EUROCRYPT)*, 2026.

[45] W. Kim, C. Lee, J. Lee, A. Passelègue, and D. Stehlé, "Asynchronous Lagrange-based threshold FHE with smaller modulus overhead," in *Proc. Annu. Cryptol. Conf. (CRYPTO)*, 2026, pp. 1–32.

[46] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 1709–1720.

[47] H. Bao, P. Chen, Y. Sun, and Z. Li, "EFSkip: A new error feedback with linear speedup for compressed federated learning with arbitrary data heterogeneity," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, 2025, pp. 15 489–15 497.

[48] P. Richtárik, E. Gasanov, and K. Burlachenko, "Error feedback reloaded: From quadratic to arithmetic mean of smoothness constants," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[49] N. Dorfman, K. Yadav, R. Camero, A. Bekkerman, and I. Hubara, "DoCoFL: Downlink compression for cross-device federated learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 8356–8386.

[50] F. Haddadpour, M. M. Kamani, A. Mokhtari, and M. Mahdavi, "Federated learning with compression: Unified analysis and sharp guarantees," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2021, pp. 2350–2358.

[51] A. Kumar, K. Singhal, A. K. Sahu, and P. Vepakomma, "Non-IID aware sparse federated learning via percentile-driven budgeting," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 18 043–18 064.

[52] M. Kim, W. Saad, M. Debbah, and C. S. Hong, "SpaFL: Communication-efficient federated learning with sparse models and low computational overhead," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 86 500–86 527.

[53] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, "Sparsified SGD with memory," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 31, 2018, pp. 4447–4458.

[54] M. Li, Y. Tian, J. Zhang, J. Xiong, X. Cui, and J. Ma, "Enhancing federated learning in edge computing with secure aggregation and dynamic verification," *IEEE Trans. Mobile Comput.*, 2025.

[55] R. Liu, Y. Cao, H. Chen, R. Guo, and M. Yoshikawa, "FLAME: Differentially private federated learning in the shuffle model," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, 2021, pp. 8688–8696.

[56] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2017, pp. 1273–1282.

[57] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[58] L. Deng, "The MNIST database of handwritten digit images for machine learning research," *IEEE Signal Process. Mag.*, vol. 29, no. 6, pp. 141–142, 2012.

[59] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2017.

[60] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.

[61] D. D. Clark and S. Wedeman, "Measurement, meaning and purpose: Exploring the M-Lab NDT dataset," in *Proc. Res. Conf. Commun., Inf. Internet Policy (TPRC)*, 2021.

[62] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal, "TenSEAL: A library for encrypted tensor operations using homomorphic encryption," in *Proc. ICLR Workshop Distrib. Private Mach. Learn. (DPML)*, 2021.